REFINEMENT-BASED CONFIRMATION OF LIVENESS PROPERTIES FROM MONITOR VERDICT TRACES: A HENSEL–NEWTON APPROACH

DAVIDE BRAGETTI
SAPIENZA UNIVERSITY OF ROME
EMAIL: DAVIDE.BRAGETTI@UNIROMA1.IT
ORCID: 0009-0008-7645-8241

Abstract. A runtime monitor that observes a finite prefix of an execution can often refute a property but never confirm it, or the reverse, and for some properties it can do neither. We study what an external structure attached to such a monitor has to supply in order to recover the missing verdict, and we introduce a refinement system for that purpose: a chain of candidate configurations, a computable update rule, and a computable error metric that measures how far the current configuration is from certifying the target. When the error metric decreases by a fixed positive quantum at every step at which it is positive, the target is certified within a bound determined by the initial error and that quantum.

The main point of the analysis is a separation the argument makes visible. Finite descent yields a statement about a prefix. Turning it into a statement about the infinite execution requires two further ingredients, stated and used separately: that the observed execution belongs to a declared class of admitted behaviours, and, for the persistent form, that the certifying condition once reached is never lost. Neither follows from the descent.

Modular inversion by Hensel lifting is the worked instance: the residual squares at each step, so a certified lower bound on the initial precision yields a guaranteed deadline for confirmation, and the confirming condition is absorbing, which supplies the temporal lift in full. Five primality methods are placed against the same scheme as a scope study. Enforcement by adapting monitor parameters rather than editing the observed trace is defined and left open.

1. Introduction

The runtime-verification picture organised by the companion work [8] classifies ω-trace properties into a sixteen-cell exclusive partition under combined topological and prefix-quantifier membership; that partition is equivalent, modulo two degenerate singletons, to the nine-cell verdict-axis partition of Peled and Havelund [9]. We refer to the sixteen cells as the CLASP cells, after the four axes Cosafety, Liveness, sAfety and coliveness that generate them. Since the rest of the paper uses this vocabulary throughout, we fix it here in the form we need; a reader who knows the classical taxonomy will recognise all of it.

Fix an alphabet $\mathcal{E}$ and identify a property with a set $\Phi \subseteq \mathcal{E}^\omega$. For a finite word $u \in \mathcal{E}^*$, say that u is a *bad prefix* for Φ if $u\sigma \notin \Phi$ for every $\sigma \in \mathcal{E}^\omega$, and a *good prefix* if $u\sigma \in \Phi$ for every σ; these are the notions of Kupferman and Vardi [33], and both quantify over *all* continuations. The four axes are then:

- S (*safety*): every $\tau \notin \Phi$ has a bad prefix — equivalently Φ is closed;
- C (*cosafety*, also called *guarantee*): every $\tau \in \Phi$ has a good prefix — equivalently Φ is open;
- L (*liveness*): every finite word extends to some $\tau \in \Phi$ — equivalently Φ is dense, so no bad prefix exists;
- K (*coliveness*): every finite word extends to some $\tau \notin \Phi$ — equivalently Φ has empty interior, so no good prefix exists.

Received by the editors 2026.

© D. Bragetti
Creative Commons

Safety and liveness are Alpern and Schneider's [31]; cosafety completes the pair, and coliveness completes the symmetry. The classes are *not* exclusive as axes — a property may be both S and K — which is why the classification is by intersection cells rather than by a single label.

Four cells occur in this paper and no others. A property in $\mathsf{S} \cap \mathsf{K}$ admits a finite *refutation* verdict but no finite confirmation; this is the reading under which "$\mathbf{\Pi}_1^0$" is used below. A property in $\mathsf{C} \cap \mathsf{L}$ admits the symmetric exchange, confirmation but no refutation. A property in $\mathsf{S} \cap \mathsf{C}$ is *clopen* and admits both. A property in $\mathsf{L} \cap \mathsf{K}$ admits neither verdict on any finite prefix; the recurrence schema $\mathbf{GF}\,\varphi$ and the persistence schema $\mathbf{FG}\,\varphi$ both live there, since any finite prefix extends both to a trace satisfying the schema and to one violating it. That cell is the paper's principal target.

One warning about the good-prefix notion, because the paper later needs a weaker one. The definition above quantifies over all continuations without restriction. Definition 3.2 introduces a version relative to a declared class of admitted traces, which is strictly weaker and is *not* the classical notion; the two coincide exactly when the admitted class is all of $\mathcal{E}^\omega$.

The gap addressed by this paper. Mathematical proof is the canonical move from refutation-only to two-sided settlement *from outside* the finite-monitor framework: the proof of Fermat's Last Theorem supplied the positive verdict on a property whose finite-monitor reading could emit only $\perp$ or "inconclusive". Most monitorable infinitary claims do not admit such a heroic external proof. This paper formalises a weaker but constructive alternative: *finite confirmation via structured supporting evidence.* A different alternative is to weaken the verdict rather than supply evidence: Esparza and Fischer replace the hard verdict by a probabilistic prediction with a confidence score that is eventually correct and grows without bound [4]. The two are complementary and not comparable in strength: theirs is a stochastic guarantee about a prediction, ours a deterministic deadline on a certificate, and neither subsumes the other. The Miller primality test under the Generalised Riemann Hypothesis is the canonical instance. "n is prime" is decidable — trial division to $\lfloor\sqrt{n}\rfloor$ settles it, unconditionally — but a monitor that enumerates candidate divisors with no a-priori bound reads it as Π_1^0 and can only emit "possibly prime" or "composite". The Miller test plus GRH supplies an external structured argument that gives that monitor a confirmation verdict at polynomial cost. What is lifted is the cost of confirmation, not the decidability of the predicate; Remark 8.1 states the scope of this reading in full.

Central thesis and separations. The paper's single central claim is that an *external* refinement structure, attached to a target whose monitor yields only one verdict, can supply a finite verdict on the augmented monitoring problem $(\Phi, \mathcal{R})$ without altering Φ's intrinsic CLASP cell. Concretely we insist on the following separations, and use them uniformly through the paper: (a) *intrinsic property class* of Φ (unchanged by the lift) vs. *operational reach* of the augmented monitoring problem (extended by the lift); (b) *finite non-observation* of a witnessing verdict (a fact about a prefix) vs. *finite certification* that the witness will appear (a fact under the certificate); (c) *a verdict-stream edit* (Schneider [16]; Falcone–Fernandez–Mounier [11]) vs. *a monitor-parameter adaptation* (Definition 9.1); (d) *convergence of the refinement* $E \to 0$ vs. *truth of* Φ *on the underlying trace*, which the structural-zero guarantee licenses only through one of the three regularity sources of Section 6. None of these distinctions is new in itself; the paper's contribution is that a single external mechanism preserves all four uniformly across per-input and ω-trace targets.

The Hensel–Newton framework. We introduce a *refinement system* $\mathcal{R} = (D, A_0, U, E)$ as the abstract structure that realises this lift. The name records the shape of the update rule: successive approximations are corrected by an error term that is driven to zero, and the canonical instance of Section 7 is Hensel lifting of a modular inverse, where that correction is exact and its residual squares at every step. Treating Hensel's lifting method and a Newton method together, in the common setting of valuation rings, is classical [2]; the hyphenated name is taken from that tradition and not coined here. Both orderings are current in that literature, and we use them for different objects: *Hensel–Newton* for the refinement tuple, and *Newton–Hensel* for the lifting procedure of Section 7, after [3]. No worked example in this paper is a literal Newton iteration

on a differentiable functional. The tuple's components inhabit specific stages of the companion pipeline [8]: D and A_0 at Stage 1 (monitor configuration), U along the Stage 1→Stage 3→Stage 1 edge, and E along the Stage 3→Stage 4 edge. Three structural sources of *regularity*—intrinsic algebraic regularity of the verdict alphabet, extrinsic regularity inherited from the generating process, and active-tuning regularity constructed runtime by the refinement system itself—provide the *structural-zero guarantee* that turns asymptotic convergence $E \to 0$ into a finite-time certificate.

Positioning. This paper develops in companion to [8] and [10]. The refinement tuple is dual to the *universal attribution theorem* of [8] (Section 11.1): that theorem describes *internal* attribution of $\mathsf{L} \cap \mathsf{K}$ pullback by any verdict monitor; the refinement tuple describes *external* attribution of finite-time confirmation to a Π^0_1 property via supplied structured evidence. The augmented monitor of [10] is the canonical event-generator instance of active-tuning regularity, expanded in Section 9. In relation to the classical enforcement framework of Schneider's security automata [16] and [11], the parametric-adaptation enforcer of Definition 9.1 operates on the monitor parameter space (Stage 1 of the companion pipeline) rather than on the verdict trace (Stage 3); the homomorphism analysis of Section 11.7 makes this distinction precise.

Trace-domain declaration. The trace domain of the present development is *bridge* in the sense of [8]: claims are stated about infinite traces $\tau \in \mathcal{E}^\omega$, but verdicts are delivered via finite Lyapunov certificates (Theorem 5.5) on bounded prefixes, with the structural-zero guarantee (Definition 5.4) licensing the finite-to-infinite step. The infinite-trace asymptotic stall states are what a bounded prefix cannot by itself exclude; the structural-zero guarantee certifies their absence on the orbit.

Roadmap. Section 2 fixes the pipeline vocabulary and maps the refinement tuple onto the six-stage substrate of the companion work. Section 3 defines the tuple formally. Section 4 encodes each tuple component by CLASP cell. Section 5 states the finite-confirmation theorem. Section 6 classifies the three regularity sources, each placed at a distinct stage of the pipeline. Sections 8 and 9 give the two worked examples. Sections 10 to 12 compare, connect and close.

2. Pipeline grounding

The companion manuscript [8] formalises a *six-stage signal–monitor–events–trace–property–meta* pipeline as the semantic backbone of the CLASP partition. We recall it here in compact form because the rest of the paper operates entirely within this substrate; we then state precisely which pipeline objects the Hensel–Newton refinement tuple acts on.

2.1. **The six-stage pipeline (compact recall).** A *process* $\mathcal{P}$ emits a *signal* $s\colon T \to D$ at Stage 0. A *monitor configuration* μ at Stage 1 (event-decision mechanism + verdict-alphabet cardinality $n \in \{1, 2, 3, 4, \infty\}$) discretises s on a bounded prefix into *events* $e \in \mathcal{E}_\mu$ at Stage 2. Aggregating events yields the *ω-trace* $\tau \in \mathcal{E}_\mu^\omega$ at Stage 3. Candidate *trace properties* $\Phi \subseteq \mathcal{E}_\mu^\omega$ are classified into the CLASP partition at Stage 4. A *meta-property* Ψ about the signal s (or the process $\mathcal{P}$) sits at Stage 5 and is *logically prior* to the choice of μ at Stage 1.

Two structural facts of the companion pipeline are essential for the present paper:

- *Stage–2 finitarity.* Events at Stage 2 are confined to the finitary CLASP cells ($\mathsf{S} \cap \mathsf{C}$, S-only, C-only) because the event-decision mechanism reads a bounded prefix of the signal. Infinitary character ($\mathsf{L}, \mathsf{K}, \mathsf{L} \cap \mathsf{K}$) emerges only at Stage 4.
- *Cross-axis alphabet constraint.* The verdict alphabet V_{μ_k} of the monitor at layer k is the per-tick alphabet $\mathcal{E}_{\mu_{k+1}}$ of the trace at the meta-layer $k+1$ that the bootstrap construction promotes τ_k to (i.e., $T3_{k+1} = D1_k = V_{\mu_k}$).

2.2. **Where the refinement tuple lives.** The Hensel–Newton tuple (D, A_0, U, E) is a construction that operates simultaneously on three of the pipeline's stages and one edge:

- $D = \{D_k\}$, *refinable computable domains*, is the sequence of admissible monitor-configuration states. In the parametric-monitor instance of Section 9, $D_k = \Pi$ (the monitor's parameter space at iteration k). **Pipeline locus: Stage 1.**
- $A_0 \in D_1$, the *coarse initial approximation*, is the starting monitor configuration before any refinement. **Pipeline locus: Stage 1 at the boot of layer 0.**
- $U\colon D_k \to D_{k+1}$, the *update rule*, is the refinement step. When the rule consumes the verdict-trace prefix emitted so far, U traverses Stage 1→Stage 3 →Stage 1 in each iteration. When the rule consumes only the previous configuration, it stays within Stage 1. **Pipeline locus: edge Stage 1→Stage 3→Stage 1.**
- $E\colon D_k \to \mathbb{R}_{\geq 0}$, the *Lyapunov error metric*, measures the distance between the current trace prefix and the satisfaction set of the target property Φ at Stage 4. E is therefore a Stage 3→Stage 4 evaluator. **Pipeline locus: edge Stage 3→Stage 4.**

Conjecture 2.1 (Refinement-tuple pipeline locus). For every Hensel–Newton refinement system (D, A_0, U, E) targeting a trace property Φ at Stage 4 of the companion pipeline, the loci above exhaust the system's pipeline footprint. In particular, no component of the tuple acts on Stage 0 (the signal) or on Stage 2 (event generation) directly: signal access is mediated entirely through the monitor at Stage 1, and event generation is determined by the current Stage 1 configuration. We do not prove the exhaustiveness claim; it is stated as a conjecture about the tuple's pipeline footprint.

2.3. **The three regularity sources are stage-located.** The three regularity sources introduced informally in the abstract each correspond to a distinct pipeline stage:

- *Intrinsic* (V, A) *regularity* (canonical instances: Miller–GRH, AKS, Pratt-tree, Mills as boundary) is supplied by the algebraic structure of the verdict alphabet V_{μ_k} and the parameter algebra of μ_k. This is a meta-property of the monitor itself, recorded at Stage 5.
- *Extrinsic regularity* is inherited from the generating process $\mathcal{P}$ behind the signal s. This is a meta-property of $\mathcal{P}$ / s, recorded at Stage 5.
- *Active-tuning regularity* is produced by the refinement system itself: parameters are controlled online to drive the verdict trace toward satisfaction of a target Layer-2 property. Active-tuning regularity is therefore the only one of the three that is *constructed* at Stage 1 rather than given at Stage 5; it is the runtime counterpart of the static regularity sources.

2.4. **Bootstrap interpretation.** When the target property Φ inhabits an infinitary CLASP cell ($\mathsf{L}, \mathsf{K}, \mathsf{L}\cap\mathsf{K}, \mathsf{L}$-only,$\mathsf{K}$-only, Quaestio), the companion pipeline offers two canonical responses: an advanced ω-monitor (e.g., Büchi-style), or the recursive bootstrap $\tau_k = s_{k+1}$ that lifts the trace at layer k to a signal at meta-layer $k+1$. The Hensel–Newton refinement tuple realises a *third* response: it leaves the layer fixed and instead supplies external structured evidence (the tuple itself) that lifts a Stage 4 verdict from "refutation-only" to "finite confirmation". The cross-axis alphabet constraint $T3_{k+1} = D1_k = V_{\mu_k}$ of the companion pipeline is preserved trivially because no meta-layer is constructed.

Remark 2.2 (Refinement as an in-layer bootstrap). The refinement tuple's relation to the companion bootstrap is that of an *in-layer* versus *across-layer* move. The across-layer bootstrap promotes τ_k to s_{k+1}; the refinement tuple keeps the layer fixed and instead promotes the verdict from refutation-only to finite-confirmation by exhausting the Lyapunov error in finite time. Both moves convert an infinitary cell verdict into a finite-time decision; they constitute the two formal options the companion framework exposes.

3. The Hensel–Newton refinement tuple

We fix the formal definition of the tuple together with three convergence criteria. Pipeline loci are recorded by Conjecture 2.1; the present section addresses the algebraic content.

Definition 3.1 (Refinement system)**.** A *refinement system* for a target trace property $\Phi \subseteq \mathcal{E}^\omega$ at Stage 4 of the companion pipeline is a tuple $\mathcal{R} = (D, A_0, U, E)$ where:

(1) $D = \{D_k\}_{k\in\mathbb{N}}$ is a sequence of *refinable computable domains* of candidate Stage 1 monitor configurations (or, more generally, of internal states of the refinement system); $D_k \subseteq D_{k+1}$ under extension semantics, $D_k \twoheadrightarrow D_{k+1}$ under quotient semantics.
(2) $A_0 \in D_1$ is a *coarse initial approximation*, the starting configuration of the refinement.
(3) $U\colon D_k \to D_{k+1}$ is a *computable update rule* (refinement step).
(4) $E\colon D_k \to \mathbb{R}_{\geq 0}$ is a *computable error metric* measuring the distance between the current configuration A_k and the satisfaction set of Φ as evaluated through the Stage 3 trace produced under A_k.

Definition 3.2 (Admitted trace class)**.** An *admitted trace class* for Φ is a set $\mathcal{T} \subseteq \mathcal{E}^\omega$ of ω-traces regarded as the behaviours the system under observation can actually produce. Writing $u\sigma$ for the concatenation of a finite prefix u with an ω-suffix σ, a finite prefix u is a *good prefix for* Φ *relative to* $\mathcal{T}$ if

$$u\sigma \in \mathcal{T} \implies u\sigma \in \Phi \qquad \text{for every } \sigma \in \mathcal{E}^\omega.$$

Taking $\mathcal{T} = \mathcal{E}^\omega$ recovers the unrelativised good prefix of [33]: every extension whatsoever satisfies Φ. Smaller $\mathcal{T}$ encodes an assumption about the observed system, in the sense of assumption-based runtime verification [5, 7]; the assumption is what a finite prefix is allowed to lean on, and it must be declared with the refinement system rather than left implicit.

Definition 3.3 (Convergence criteria)**.** Write $A_k := U^k(A_0) \in D_{k+1}$ for the k-fold update from A_0. The refinement system $\mathcal{R}$ is:

- *strongly convergent for* Φ if $\exists k^* \in \mathbb{N}$ such that $E(A_{k^*}) = 0$ (finite-time exact certificate);
- *ε-convergent for* Φ at tolerance $\varepsilon > 0$ if $\exists k^* \in \mathbb{N}$ such that $E(A_{k^*}) \leq \varepsilon$;
- *asymptotically convergent for* Φ if $\lim_{k\to\infty} E(A_k) = 0$;
- *divergent (failure)* otherwise.

The system *certifies* Φ at level k^* if it is strongly convergent at k^*.

Definition 3.4 (Refinability and computability)**.** The pair (D, U) is *refinable* if D_{k+1} admits strictly more discriminative structure than D_k (the orbit $\{U^k(A_0)\}_k$ is non-stationary unless $\mathcal{R}$ has certified Φ). The triple (D, U, E) is *computable* in a fixed model of computation if U and E are total recursive functions.

Remark 3.5 ((V, A) parameter-pair pattern)*.* The Hensel–Newton tuple instantiates a broader parameter-pair pattern (V, A) recurring across the companion line, where V is a value system (verdict alphabet, candidate-domain set, or number-system carrier of the property) and A is an algebra of operations on V. The split aligns as follows: V as the sequence of refinable candidate domains $\leftrightarrow$ $D = \{D_k\}$; initial element of $V \leftrightarrow A_0$; A on V (refinement direction) $\leftrightarrow U$; A on V (distance to target) $\leftrightarrow E$. The same pattern recurs in [8] with V varying along the verdict-cardinality ladder $n \in \{2, 3, 4, \infty\}$ at fixed A, and in [10] with V fixed at the ternary event alphabet $\{\mathsf{V}, \mathsf{I}, \varepsilon\}$ and A varying along touch/crossing duality. The recurrence explains the structural questions that connect the three lines of work; the recurrence is not claimed as a formal unification.

Proposition 3.6 (Cost of the update step, three instances)**.** *For $i \in \{$sieve, Miller, AKS$\}$ the update rule U_i and the error metric E_i of Section 8 are computable in time polynomial in $\log n$, the representation size of the candidate: one sieve constraint costs $O(k \log n)$; one Miller round is a modular exponentiation, $\widetilde{O}((\log n)^2)$ [17]; and the AKS decision procedure runs in $\widetilde{O}((\log n)^6)$ in the Lenstra–Pomerance form* [19]*. The bounds are those of the cited algorithms; nothing is proved here beyond the observation that U_i and E_i are the algorithms' own steps.*

Remark 3.7 (Pratt and Mills are outside the statement, for different reasons). Four costs must be kept apart: the *size* of a certificate, the cost of *verifying* it, the cost of *constructing* it, and the cost of *deciding* the predicate without one. Pratt's tree has size $O(\log^2 n)$ and is verifiable in $\widetilde{O}((\log n)^3)$ bit operations [20]; its *construction*, however, requires the complete factorisation of $n-1$ at every node, for which no polynomial-time algorithm is known. An earlier version of this paper reported the size bound as a construction bound, which was an error; the obstacle is factoring, not the non-uniformity of the tree shape. Mills's construction is excluded on a different ground: it is non-effective, since the constant must be supplied in advance (Section 8.5), so no computability bound on U is available to state.

Remark 3.8 (Refinement vs trace observation). A standard runtime monitor is the passive map $M\colon \mathcal{E}^* \to V$ that observes the trace prefix emitted by a fixed monitor configuration. The refinement system of Definition 3.1 is, by contrast, an *active* construction that adjusts the configuration $A_k \in D_k$ in response to the trace prefix accumulated so far. The two roles are complementary: a refinement system requires a monitor in order to observe what its updates produce, and a monitor admits a refinement system as soon as its configuration is exposed as a computable parameter space.

The structural-zero guarantee promised in Definition 3.3 as the precondition for strong convergence is articulated in Section 6 as the consequence of one of three regularity sources, each placed at a specific pipeline locus.

4. Property-class encoding

Each component of the refinement tuple (Definition 3.1) encodes a specific property class of the companion partition. We present the encoding as a two-tier table: (i) per-component CLASP-cell role; (ii) Floor-N hierarchy stating which composite configurations of the tuple lift a property from one floor to a strictly higher one.

Component	Role	CLASP-cell role encoded
$D = \{D_k\}$	unbounded refinement space	L (progress always possible)
$A_0 \in D_1$	controlled initial state	$\mathsf{S/C}$ (start in known good zone)
$U\colon D_k \to D_{k+1}$	computable refinement engine	strategic enforceability + constructive L
$E\colon D_k \to \mathbb{R}_{\geq 0}$	observable error metric	monitorability + C + L

Table 1: Per-component CLASP-cell encoding of the refinement tuple. When all four components are present and computable, the combination realises a *Floor 6 lift*: the system begins in a $\mathsf{S/C}$ zone via A_0, pursues L via the refinement chain D, exposes enforceability via the computable update U, and supports C via the error metric E at the $E = 0$ certificate.

4.1. **Per-component encoding.**

4.2. **Floor-N hierarchy.**

The companion partition is a horizontal classification: cells are mutually exclusive on the level of properties of $\mathcal{E}^\omega$. The refinement framework superimposes a vertical *Floor-N hierarchy* that records what monitor *plus* refinement-tuple configuration achieves which operational reach. A property's intrinsic CLASP cell is unchanged by the lift; the floor records the external scaffolding.

Corollary 4.1 (Floor-N lift)**.** *For a property Φ in CLASP cell S-only (Floor 4 in Table 2), suppose a refinement system $\mathcal{R} = (D, A_0, U, E)$ satisfies the hypotheses of Theorem 5.5 — in particular E decreases by a* uniform *quantum $c > 0$ while positive. Then there is $k^* \leq \lceil E(A_0)/c \rceil$ with $E(A_{k^*}) = 0$, and the* augmented monitoring problem $(\Phi, \mathcal{R})$ *admits a two-sided verdict at level k^*: refutation from the bad prefix that $\Phi \in \mathsf{S}$ already supplies, confirmation from the*

Floor	CLASP cell of Φ	Tuple configuration
0	strong K	no D, no U, no E observable
1	weak K	sparse D, non-deterministic U
2	L (cell L-only)	infinite D-chain, U exists, no finite witness
3	C (cell C-only)	A_k with $E(A_k) = 0$ detectable finitarily
4	S (cell S-only)	bad prefix detectable finitarily
5	$\mathsf{S} \cap \mathsf{C}$ (clopen)	both verdicts decidable finitarily
6	**strategic enforceability**	**full tuple active, U strategic, E strictly decreasing**
≥ 7	proof-carrying closure	tuple plus formal certificate generator

Table 2: Floor-N hierarchy of operational reach under refinement. Floor 6 is the level at which a Π^0_1 property—natively Floor 4 by the CLASP partition—can be *lifted to Floor 5* ($\mathsf{S} \cap \mathsf{C}$) by the addition of an appropriate refinement tuple $\mathcal{R}$; Floor 7 lifts further to proof-carrying certificates. The lift is external: the property's intrinsic CLASP cell does not change. *Type note.* Floors 0–4 index the intrinsic CLASP cell of the target property Φ; Floors 5–7 index the *certificate configuration* attached to Φ. These are two coordinates, not one linear scale: the operational state of a target is the pair (intrinsic cell of Φ, floor of the attached system). Higher-floor entries are not stronger *properties*; they are stronger *monitoring problems* obtained by pairing Φ with progressively richer external evidence (cf. Corollary 4.1).

certificate. The intrinsic CLASP cell of Φ *remains* S*-only. The pair* (cell of Φ, floor of the system) *is the operational coordinate of the lifted verdict; the two coordinates do not compose into a single linear order (Table 2 type note).*

Proof. Immediate from Theorem 5.5: its conclusion is exactly $E(A_{k^*}) = 0$ at some $k^* \leq \lceil E(A_0)/c \rceil$, and the target-reach clause makes $u_{A_{k^*}}$ a good prefix relative to $\mathcal{T}$, which is the confirmation verdict. □

Remark 4.2 (Why the uniform quantum cannot be dropped here). The hypothesis is stated as *uniform* decrease deliberately. Under the weaker hypothesis that E merely decreases strictly along the orbit, the conclusion fails: on $D_k = \mathbb{N}$, $U(k) = k + 1$, $E(A_k) = 2^{-k}$, the metric is non-negative and strictly decreasing, and the structural-zero guarantee of Definition 5.4 is granted vacuously (the orbit does not satisfy the fixed-c clause, so it is not among the orbits quantified over), yet $E(A_k) > 0$ for every k and no k^* exists. This is the same gap that Proposition 5.2 closes for the uniform case.

5. Main theorem: finite confirmation under bounded Lyapunov refinement

The central claim is that strong convergence of the refinement system is reachable in a bounded number of steps when the error metric obeys a Lyapunov-style strict-decrease property and a structural-zero guarantee precludes asymptotic stagnation just above zero.

Definition 5.1 (Lyapunov error metric with strict decrease)**.** The error metric E of Definition 3.1 is a *Lyapunov metric with strict-decrease constant* $c > 0$ on target Φ *relative to an admitted trace class* $\mathcal{T}$ (Definition 3.2) if:

- (non-negativity) $E(A) \geq 0$ for every $A \in \bigcup_k D_k$;
- (target reach relative to $\mathcal{T}$) $E(A) = 0$ if and only if the verdict-trace prefix u_A produced under A is a good prefix for Φ relative to $\mathcal{T}$, that is, $u_A\sigma \in \mathcal{T}$ implies $u_A\sigma \in \Phi$ for every $\sigma \in \mathcal{E}^\omega$;
- (strict decrease on non-zero) for every A_k with $E(A_k) > 0$, $E(A_{k+1}) \leq E(A_k) - c$.

The quantifier in the target-reach clause is the load-bearing one and is stated explicitly for that reason. The unrelativised reading $\mathcal{T} = \mathcal{E}^\omega$ is the strongest and is what the Newton–Hensel instance of Section 7 satisfies. Replacing it by the existential reading "some extension of u_A satisfies Φ" would make the framework unsound: for $\Phi = \mathbf{GF}\, p$ over $\{p, \neg p\}$, the metric $E(u) = \max(0, 1 - \#\{p \text{ in } u\})$ is non-negative, decreases by $c = 1$ while positive, and reaches 0 on the prefix p, whose continuation $p\,\neg p\,\neg p \cdots$ does not satisfy Φ. That reading is therefore excluded.

Proposition 5.2 (Uniform progress-quantum: orbit gap)**.** *Under Definition 5.1, on any refinement orbit $\{A_k\}$ the metric satisfies*

$$E(A_k) \in \{0\} \cup [c, \infty) \qquad \text{whenever } A_k \text{ has a successor } A_{k+1} = U(A_k) \text{ on the orbit.}$$

That is, the positive values taken by E on the orbit are bounded away from zero by the uniform progress quantum c.

Proof. Suppose $E(A_k) > 0$. By the strict-decrease clause, $E(A_{k+1}) \leq E(A_k) - c$. By the non-negativity clause, $E(A_{k+1}) \geq 0$. Combining, $0 \leq E(A_k) - c$, so $E(A_k) \geq c$. Contrapositively, $E(A_k) \in \{0\} \cup [c, \infty)$. □

Remark 5.3 (Reading of Proposition 5.2). The gap property is a *definitional consequence*, not an additional hypothesis: it follows from combining strict-decrease (with fixed c) and non-negativity, so any triple (E, U, c) satisfying Definition 5.1 automatically respects the gap on the orbit. Two implications matter downstream: (i) the "Lyapunov" analogy of the framework is stronger than the usual continuous-analysis Lyapunov argument — it presupposes a uniform *progress quantum* $c > 0$, not merely a monotone descent to zero; (ii) the general existence problem for E (Remark 5.11) is correspondingly harder — one must construct an E whose orbit values never enter the open interval $(0, c)$, not merely an E that decreases asymptotically to zero.

Definition 5.4 (Structural-zero guarantee)**.** A refinement system $\mathcal{R}$ admits a *structural-zero guarantee* for Φ if there is no infinite orbit $\{A_k\}_{k \in \mathbb{N}}$ with $E(A_k) > 0$ for all k on which the strict-decrease property of Definition 5.1 holds. Regularity sources supplying this guarantee are catalogued in Section 6.

This is a condition on the *orbit of E* and on nothing else. It is not, and must not be read as, a licence to pass from a finite witnessing prefix to a guarantee about the underlying ω-trace; that second and logically independent step is carried by the admitted trace class $\mathcal{T}$ of Definition 3.2 and is discharged by Proposition 5.6. Earlier drafts of this section used one name for both, which is a conflation we now avoid.

We also record what the condition does *not* say. It is not equivalent to "the orbit cannot stall at a strictly positive value of E". The constant orbit $E \equiv 1$ stalls at 1, yet does not satisfy the strict-decrease clause and so is not one of the orbits quantified over: such a system vacuously admits the guarantee while stalling. The two formulations diverge exactly on orbits that fail strict decrease, which is the regime Remark 5.8 reserves the hypothesis for.

Theorem 5.5 (Finite confirmation under bounded Lyapunov refinement)**.** *Let $\mathcal{R} = (D, A_0, U, E)$ be a refinement system for a target property Φ, with admitted trace class $\mathcal{T}$, satisfying:*

(a0) *(D, U, E) is computable in the sense of Definition 3.4; in particular U is total, so the orbit $A_k := U^k(A_0)$ is defined for every k;*

(a) *E is a Lyapunov error metric with strict-decrease constant $c > 0$ relative to $\mathcal{T}$ (Definition 5.1);*

(b) *the initial error is bounded by $E(A_0) \leq E_{\max} < \infty$;*

(c) *$\mathcal{R}$ admits a structural-zero guarantee (Definition 5.4).*

Then, with $N := \lceil E(A_0)/c \rceil \leq \lceil E_{\max}/c \rceil$,

$$\exists k^* \leq N \quad \text{such that} \quad E(A_{k^*}) = 0,$$

and consequently the verdict-trace prefix $u_{A_{k^*}}$ is a good prefix for Φ relative to $\mathcal{T}$.

The conclusion is a statement about a finite *prefix. It does not by itself assert that the observed ω-trace satisfies Φ: that requires in addition that the trace lie in $\mathcal{T}$, and, for the persistence form, the absorption hypothesis of Proposition 5.6. Hypothesis (a0) is not cosmetic: without totality the orbit may simply stop. Take $D = \{a_0, a_1\}$ with $U(a_0) = a_1$ and U undefined at a_1, $E(a_0) = 1.5$, $E(a_1) = 0.5$, $c = 1$; every stated clause holds on the orbit points that have successors, $N = 2$, and yet E never vanishes.*

Proof. Suppose for contradiction that $E(A_k) > 0$ for every $k \in \{0, 1, \ldots, N\}$. Then by the strict-decrease clause of Definition 5.1, applied at each $k \in \{0, \ldots, N-1\}$, we have $E(A_{k+1}) \leq E(A_k) - c$, and iterating from A_0:

$$E(A_N) \;\leq\; E(A_0) - Nc \;\leq\; E(A_0) - \lceil E(A_0)/c \rceil \cdot c \;\leq\; 0,$$

because $\lceil E(A_0)/c \rceil \cdot c \geq E(A_0)$. By the non-negativity clause of Definition 5.1, $E(A_N) \geq 0$, so $E(A_N) = 0$, contradicting the assumption $E(A_N) > 0$. Therefore there exists $k^* \in \{0, 1, \ldots, N\}$ such that $E(A_{k^*}) = 0$. Hypothesis (a0) guarantees that each $A_{k+1} = U(A_k)$ used above exists. By the target-reach clause of Definition 5.1, $E(A_{k^*}) = 0$ gives that $u_{A_{k^*}}$ is a good prefix for Φ relative to $\mathcal{T}$ — the certification of the augmented monitoring problem $(\Phi, \mathcal{R})$ in the sense of Definition 3.3.

Hypothesis (c) is not used in the proof; it is subsumed by (a)+(b) at the strict-decrease clause's current strength, as recorded in Remark 5.8. Hypothesis (b) is likewise not needed for the sharper bound: E is real-valued, so $E(A_0) < \infty$ holds by typing, and (b) earns its place only when $E_{\max}$ is a bound uniform over a class of initial approximations. The proof shows that the sharper bound $\lceil E(A_0)/c \rceil$ holds; the coarser bound $\lceil E_{\max}/c \rceil$ follows from $E(A_0) \leq E_{\max}$. □

Proposition 5.6 (From finite confirmation to a temporal guarantee)**.** *Let $\mathcal{R}$, Φ, $\mathcal{T}$ and k^* be as in Theorem 5.5, and let $\tau \in \mathcal{T}$ be an observed trace extending $u_{A_{k^*}}$. Let C be the predicate on orbit points defined by $C(A) :\Leftrightarrow E(A) = 0$.*

(i) (Eventuality.) *$\tau \models \Phi$, and C holds at some index $k^* \leq N$; hence $\tau \models \mathbf{F}\, C$, witnessed by a good prefix of length at most N.*

(ii) (Persistence.) *If in addition C is* absorbing *along the orbit — $C(A_k)$ implies $C(A_{k+1})$ for every k — then $\tau \models \mathbf{FG}\, C$.*

(iii) (Recurrence.) $\mathbf{GF}\, C$ *does* not *follow from (i) or (ii). It requires a re-entry hypothesis: an infinite set $K \subseteq \mathbb{N}$ of indices at which the refinement is re-initialised to a state with $E > 0$ and the hypotheses of Theorem 5.5 are re-established. Absent such a hypothesis no recurrence claim is licensed.*

Proof. (i) By Theorem 5.5, $E(A_{k^*}) = 0$, so $u_{A_{k^*}}$ is a good prefix for Φ relative to $\mathcal{T}$. Since $\tau \in \mathcal{T}$ extends $u_{A_{k^*}}$, the defining implication of Definition 3.2 gives $\tau \in \Phi$. That C holds at $k^* \leq N$ is the theorem's conclusion, and goodness is upward closed — every extension of a good prefix is good — so the witness length is at most N and no minimality is claimed.

(ii) Absorption gives $C(A_k)$ for every $k \geq k^*$ by induction on k, with base $C(A_{k^*})$ from (i). So all but finitely many positions satisfy C, which is $\mathbf{FG}\, C$.

(iii) The two hypotheses are consistent with a trace on which C holds from k^* onward and never fails again; on such a trace $\mathbf{GF}\, C$ holds only in the degenerate way that $\mathbf{FG}\, C$ already forces, and no infinitely repeated *fresh* confirmation occurs. Conversely, absorption is incompatible with the alternation that a non-degenerate recurrence claim reports. Hence recurrence needs the stated re-entry hypothesis and is not a consequence of finite descent. □

Remark 5.7 (What each hypothesis buys). The three temporal forms are produced by three different ingredients and should not be traded for one another. Finite descent (Theorem 5.5) yields a bounded *hitting* statement about a prefix. Membership of the observed trace in $\mathcal{T}$ converts that prefix statement into Φ on the trace, and is an assumption about the observed system, not a theorem about the refinement. Absorption of C upgrades **F** to **FG**. Recurrence requires re-entry and is not delivered by any of the three. Proposition 7.3 is the instance of clause (ii) that the Newton–Hensel system of Section 7 discharges in full.

Remark 5.8 (Structural-zero as a scope condition, not an independent hypothesis, under the current Definition 5.1). Under the strict-decrease clause of Definition 5.1 as literally stated—"for every A_k with $E(A_k) > 0$, $E(A_{k+1}) \leq E(A_k) - c$" with a fixed $c > 0$—and the bounded initial error hypothesis (b) of Theorem 5.5, the structural-zero guarantee (c) is automatically satisfied. The cleanest way to see this is via Proposition 5.2: positive orbit values are bounded below by c, so telescoping the strict-decrease clause over $\lceil E_{\max}/c \rceil$ steps drives E into $\{0\} \cup [c, \infty)$ with a total drop of at least $\lceil E_{\max}/c \rceil \cdot c \geq E_{\max}$; combined with non-negativity, the orbit hits zero at some $k^* \leq \lceil E_{\max}/c \rceil$. Hypothesis (c) is therefore listed for a distinct role: it is the SCOPE CONDITION that this framework does not apply to weakened decrease notions ("monotone non-increase with strict decrease at infinitely many steps") under which stuttering—repeated updates that leave E unchanged—is admitted. A future generalisation that replaced the strict-decrease clause by an eventual-decrease clause would make (c) a genuine additional hypothesis. Which worked examples actually instantiate the strong clause is a question to be settled case by case, and the answer is not uniform. The Newton–Hensel system of Section 7 does: Section 7 exhibits the constant $c = 1$ and verifies the decrease on both branches. The primality systems of Section 8 do so only with the residual metrics adopted there (Remark 8.2); the accumulated-evidence readings they replace are non-decreasing and do not instantiate the clause at all.

Remark 5.9 (Stuttering and the bridge-domain step). Two reader-safety points that the current statement of Theorem 5.5 leaves implicit. *(i) Stuttering.* If the update rule U is idempotent on some A_k with $E(A_k) > 0$ (that is, $U(A_k) = A_k$), the strict-decrease clause of Definition 5.1 is violated at that step and the theorem's hypothesis (a) does not apply—not silently, but by the strict form of the clause. A reader expecting stuttering to be absorbed by the theorem should note that it is not: stuttering is either excluded by the tuple's construction — as in Section 8, where each U-step consumes one further sieve constraint, base or congruence, so no step can leave E unchanged — or ruled by Remark 5.8's scope condition. *(ii) Bridge-domain step.* Theorem 5.5 concludes $E(A_{k^*}) = 0$, and Definition 5.1's target-reach clause identifies $E(A) = 0$ with the existence of a finite verdict-trace prefix witnessing Φ. The step from "finite witnessing prefix" to "guarantee on the underlying ω-trace" is the bridge-domain move declared in Section 1's trace-domain paragraph and licensed—per target—by one of the regularity sources of Section 6. Pure L or K targets without a regularity source do not admit the bridge on Theorem 5.5's conclusion alone.

Remark 5.10 (Trace directionality: producer / evaluator / progress). Definition 3.1's wording "E measures the distance between A_k and the satisfaction set of Φ as evaluated through the Stage-3 trace *produced under* A_k" fixes a direction of dependency that is worth naming: the configuration A_k is the *producer* of the trace τ_k, and the property Φ is the *evaluator* applied to that trace to yield the progress measure.

$$A_k \xrightarrow{\text{execute}} \tau_k \xrightarrow{\text{evaluate}_\Phi} E(A_k)$$

This is the opposite direction from the standard runtime-monitor shape $(\text{monitor}, \tau) \mapsto \text{verdict}$ in which the trace is an *input* parameter alongside the monitor. The Hensel–Newton refinement system uses the configuration as the generator of the very trace it is evaluated against; the trace is a *derived* object, not an independent argument. Any interface that would try to represent the system with a trace-as-input signature would invert this dependency and lose the framework's central move. This is the distinction between a *configuration-produced trace* (HN) and an *externally-supplied trace* (classical RV).

Remark 5.11 (Where the scientific weight lives). Theorem 5.5 is arithmetically light: given the assumptions of Definition 5.1 it is a standard finite-descent argument on a non-negative sequence with uniform strict decrease. The framework's substantive burden lies elsewhere: (i) in the target-reach clause of Definition 5.1, which is a *definition-level obligation* on the constructor of E ("$E(A) = 0 \Leftrightarrow A$ admits a finite verdict-trace prefix witnessing Φ"); and (ii) in the existence question that Section 6 addresses, "for which non-trivial Φ can such an E actually be

exhibited?". The five primality worked examples of Section 8 and the Newton–Hensel system of Section 7 are answers to (ii) at instances; each is an existence proof at a point, not a general existence theorem. A general existence result across a declared class of Φ is outside the scope of the present paper and is recorded as future work in Section 12. Moreover, per Proposition 5.2, that general existence problem is strictly harder than a generic Lyapunov-descent construction suggests: one must exhibit an E whose orbit values never enter the open interval $(0, c)$, so an E that merely descends asymptotically to zero does not qualify.

Remark 5.12 (Constructiveness of the bound). The bound $\lceil E_{\max}/c \rceil$ is the standard Lyapunov-step count. In the worked examples of Section 8, the bound specialises further: Miller–GRH yields $O((\log n)^2)$ steps, AKS yields $\widetilde{O}((\log n)^6)$ steps, Pratt-tree yields $O(\log n)$ steps at the cost of carrying a generative certificate. In the parametric-monitor instance of Section 9 the bound is the dimension of the parameter space times the discretisation granularity.

6. Three regularity sources

The structural-zero guarantee of Definition 5.4 is not a property of the refinement system in isolation; it is *supplied* by background structure that either (i) the verdict alphabet V_μ and its associated operation algebra already carry, (ii) the generating process $\mathcal{P}$ behind the signal s exposes, or (iii) the refinement system itself constructs at runtime by adapting Stage 1. The three *regularity sources* below correspond to these three modes; their pipeline loci are summarised by Section 2.3.

6.1. Intrinsic (V, A) regularity.

Intrinsic regularity is carried by the algebraic structure of the pair (V, A) where V is the verdict alphabet of the monitor at Stage 1 and A is its associated operation algebra (compare [8] rem:parameter-pair). The structural-zero guarantee follows from an algebraic invariant: for instance, Miller's witness algorithm exploits the multiplicative group structure of $(\mathbb{Z}/n\mathbb{Z})^\times$ under the Generalised Riemann Hypothesis; AKS exploits cyclotomic polynomial congruences; Pratt-tree certificates exploit recursive divisibility witnesses. In each case the algebraic invariant provides a number-theoretic discriminator on which E must land in finite steps.

Definition 6.1 (Intrinsic (V, A) regularity)**.** A refinement system $\mathcal{R}$ for Φ enjoys *intrinsic* (V, A) *regularity* if there are a finite group G, a distinguished subgroup $H \leq G$, and a non-trivial map $\iota_0 \colon V \to G$ on the verdict alphabet, such that, writing $\iota \colon V^* \to G$ for the induced monoid homomorphism

$$\iota(v_1 v_2 \cdots v_k) \;=\; \iota_0(v_1)\, \iota_0(v_2) \cdots \iota_0(v_k), \qquad \iota(\varepsilon) = 1_G,$$

the following hold for every orbit, where $\tau_k \in V^*$ denotes the verdict-trace prefix produced under A_k:

(i) $E(A_k) = 0 \iff \iota(\tau_k) \in H$;

(ii) there is a bound N_ι, depending only on (G, H, ι_0), such that $\iota(\tau_k) \in H$ for some $k \leq N_\iota$.

The typing is stated because it is easy to get wrong: ι_0 acts on *letters* and ι on *words*, and it is ι that is applied to a trace prefix. An earlier version wrote a single map on V and applied it to τ_k, which is not well-formed. Clause (ii) replaces the earlier informal requirement that the orbit "cannot remain in $G \setminus H$ forever without violating the algebra of (V, A)"; as phrased, that was not a checkable condition, whereas (ii) is exactly what is needed to supply the structural-zero guarantee of Definition 5.4 and is verifiable for a given (G, H, ι_0).

We do not claim that the worked examples below exhibit (G, H, ι_0) explicitly: they do not, and constructing them is open. What §6 claims is only that the algebraic structure named in the Miller case (Section 8.2), the multiplicative group $(\mathbb{Z}/n\mathbb{Z})^imes$, is where such an invariant would have to live.

6.2. **Extrinsic regularity.** *Extrinsic* regularity is inherited from the generating process $\mathcal{P}$ behind the signal s. When $\mathcal{P}$ is itself regular—periodic, stationary, ergodic, or bounded by a known statistical model—the trace τ inherits that regularity, and the Lyapunov error metric E inherits its strict-decrease bound from $\mathcal{P}$'s mixing time or attractor-convergence rate.

Definition 6.2 (Extrinsic regularity)**.** A refinement system $\mathcal{R}$ enjoys *extrinsic regularity* (under generating process $\mathcal{P}$) if the strict-decrease constant c of Definition 5.1 admits an explicit lower bound expressible solely in terms of $\mathcal{P}$'s parameters (e.g. mixing time τ_{mix}, ergodic constant, periodicity). Equivalently, a Stage 5 meta-property about $\mathcal{P}$ already encodes the structural-zero guarantee.

Example 6.3 (Poincaré recurrence: physical liveness confirmed without observation)**.** Let a Hamiltonian gas of N particles be confined to a sealed box on a finite-energy shell Σ_E, with phase-space flow φ^t. Fix an initial microstate x_0 and a tolerance $\varepsilon > 0$, and take the target property

$$\Lambda \;=\; \mathbf{GF}\,(\text{return to } B_\varepsilon(x_0)),$$

"the trajectory returns infinitely often to an ε-neighbourhood of its initial microstate". In the CLASP partition of [8] this is the recurrence schema $\mathbf{GF}\,\varphi$: it lies in L, and in fact in the zero-sided cell $\mathsf{L} \cap \mathsf{K}$—never finitely refutable nor finitely confirmable by trace observation as established in that work.

The confirmation problem is extreme. The recurrence time of a macroscopic gas is super-exponential in N, astronomically beyond any physical observation horizon, so *no* finite verdict-trace prefix can ever witness a return: Λ is the paradigm of a liveness property that is unconfirmable by observation. Yet confirmation is licensed—not by the trace, but by *extrinsic regularity* (Definition 6.2) of the generating flow. The supporting arguments are measure-theoretic: (i) the flow φ^t preserves the Liouville measure μ, and (ii) the energy shell has finite total measure $\mu(\Sigma_E) < \infty$. By Poincaré's recurrence theorem [22] these premises certify Λ with no observation at all—historically the content of Zermelo's 1896 recurrence objection (*Wiederkehreinwand*) to Boltzmann [23], applied to exactly this gas thought-experiment.

We deliberately do *not* read this as a refinement system. An earlier version did, taking the error metric to be the measure of the non-recurrent exceptional set at a decreasing tolerance; but Poincaré's theorem forces that measure to 0 at *every* tolerance, including the initial one, so the metric is identically zero, no step ever decreases it, and the tuple performs no work. The target-reach clause fails for the same scenario from the other side, since the example's own point is that no finite prefix can witness a return.

What the example illustrates is the extreme case of extrinsic regularity: a certificate supplied *wholly* by an external theorem, with no refinement dynamics at all and no observation whatsoever. It marks the boundary of the framework rather than an instance of it, and that is why it is stated as an example and not as a proposition.

Remark 6.4 (Scope of the recurrence certificate)*.* The certificate of Example 6.3 is incomplete on two fronts, so the honest statement is "almost every initial condition recurs", not "all particles return". First, $E(A_k) = \mu(N_{\varepsilon_k}) = 0$ is an *almost-everywhere* verdict: a measure-zero non-recurrent set N_{ε_k} is excluded, never observed away. Second, the guarantee is valid only in the unbounded-time limit; the astronomical recurrence time is the gap between the finite observation horizon and the certified return. As a bridge-domain property (finite evidence, infinite trace), the certificate makes this recurrence-time / measure-zero caveat explicit rather than leaving it implicit.

6.3. **Active-tuning regularity.** *Active-tuning* regularity is the only one of the three that is *constructed* by the refinement system at runtime, rather than presupposed. When the monitor at Stage 1 is parametric (parameter space Π) the update rule $U_\varphi \colon \Pi \times V^* \to \Pi$ of Definition 9.1 adapts the parameters online to drive the verdict trace toward satisfaction of Φ. Each adaptation step refines $D_k \subseteq \Pi$ in a direction that the Lyapunov error confirms is monotone-decreasing;

the structural-zero guarantee is supplied by the monotonicity of the augmented monitor in each parameter coordinate ([10]).

Definition 6.5 (Active-tuning regularity)**.** A refinement system $\mathcal{R}$ on a parametric monitor with parameter space Π enjoys *active-tuning regularity* for Φ if there is a computable adaptation rule $U_\varphi \colon \Pi \times V^* \to \Pi$ such that the Lyapunov metric E on the orbit $\{(\text{state}_k, \tau\restriction_k)\}_k$ produced by U_φ is strictly decreasing on non-zero steps, and the augmented-monitor monotonicity precludes infinite stagnation at $E > 0$.

Remark 6.6 (Three sources are not exclusive). A single refinement system may rely on multiple regularity sources simultaneously. The Miller–GRH example combines intrinsic regularity (group-theoretic) with mild extrinsic regularity (random base selection); the parametric-monitor enforcer of Section 9 combines extrinsic regularity of the signal with active-tuning regularity of the monitor parameters.

7. Worked example: modular inversion by Newton–Hensel lifting

The worked example that follows attaches a refinement system to a target property and then classifies what the system confirms. This section does something narrower and, for the purposes of the framework, prior: it follows a single monitored execution from raw observations to a temporal statement about it, with every intermediate step proved. The arithmetic is classical—the iteration, its error dynamics, and the resulting explicit formula are due to [1]—and no arithmetic claim is made here. What the section exhibits is the workflow: a relation is *noticed* between consecutive monitor observations, *conjectured*, *proved*, and only then lifted to a global trace law and a temporal property.

The system. Fix a prime p and a with $p \nmid a$, and let b satisfy $ab \equiv 1 \pmod{p}$. The refinement step is the Newton iteration

$$N_a(x) \;=\; x\,(2 - ax), \qquad x_0 = b.$$

The admissible carrier is not the full unit group but the lifting basin

$$C_a \;=\; \{\, x : ax \equiv 1 \pmod{p} \,\},$$

which N_a preserves: if $ax \equiv 1 \pmod{p}$ then $2 - ax \equiv 1 \pmod{p}$, so $N_a(x)$ is again a unit and again lies in C_a. For $p = 2$ and odd a the condition $ax \equiv 1 \pmod{2}$ holds exactly when x is odd, so there C_a *coincides with* $U(2^n)$; the basin restriction is invisible in the binary case rather than absent from it. For odd p it is not: $N_1(2) = 0$ modulo 3 shows that N_a does not preserve $U(p^n)$ in general.

E1: what a finite run shows. Take $p = 2$, $a = 3$, $x_0 = 1$, and record alongside each state x_k the residual $r_k = 1 - ax_k$ and its valuation $m_k = \nu_p(r_k)$. The run below is computed exactly in $\mathbb{Z}$, and every entry is reproducible in a line of integer arithmetic from the recurrence $x_{k+1} = x_k(2 - ax_k)$:

k	x_k	$r_k = 1 - 3x_k$	$m_k = \nu_2(r_k)$
0	1	−2	1
1	−1	4	2
2	−5	16	4
3	−85	256	8
4	−21845	65536	16

Table 3: Five steps of N_3 from $x_0 = 1$. The state column is unremarkable; the residual column is not.

E2: the conjecture. The residual column of Table 3 reads $-2, 4, 16, 256, 65536$: each entry is the square of its predecessor, and correspondingly m_k doubles. Five samples suggest

$$r_{k+1} \;=\; r_k^2,$$

and suggest nothing more. A finite run cannot establish a law that quantifies over all k; it can only tell us which law is worth trying to prove.

E3: the proof.

Proposition 7.1 (Local trace law)**.** *For every x and every a, with $r_a(x) = 1 - ax$,*

$$r_a(N_a(x)) \;=\; r_a(x)^2.$$

Proof. $1 - a\,x(2 - ax) = 1 - 2ax + a^2x^2 = (1 - ax)^2$. □

The identity holds in $\mathbb{Z}[a, x]$, so it survives reduction modulo any p^n; the residual observable is the coordinate in which the refinement step becomes squaring.

E4: globalisation and closed form.

Corollary 7.2 (Global trace law and closed form)**.** $r_k = r_0^{2^k}$ *for every $k \geq 0$, and consequently*

$$x_k \;=\; \frac{1 - r_0^{2^k}}{a}, \qquad r_0 = 1 - ax_0.$$

Proof. Induction on k: the base case is trivial and the step is Proposition 7.1. Then $ax_k = 1 - r_k = 1 - r_0^{2^k}$. □

The whole trace is therefore recoverable from (r_0, k): the intermediate states are eliminable because the residual dynamics compose exactly. Expanding the same recurrence as a product recovers the explicit formula of [1], which is stated there for arbitrary prime powers; the agreement is external validation of the derivation above, not its source.

Coarsening, and what each resolution retains. Three observables were recorded, at decreasing resolution:

$$x_k \;\overset{r_a}{\longmapsto}\; r_k \;\overset{\nu_p}{\longmapsto}\; m_k \;\overset{\geq n}{\longmapsto}\; \mathsf{conf}_n.$$

For fixed unit a the map $x \mapsto 1 - ax$ is affine with invertible coefficient, hence bijective, and the state is recoverable from the residual as $x = (1 - r)/a$; that is why the closed form above could be read off the residual trace at all. The valuation is many-to-one and the threshold coarser still. A monitor that had recorded only the Boolean column could certify that the target was reached and could not have produced Corollary 7.2: what a trace records bounds what can later be reconstructed from it.

The chain is also where the work of this example is done, and it is worth saying so. Deciding invertibility modulo p^n directly is not the shape of a simple test; but in the residual–valuation representation the target is exactly $\nu_p(1 - ax) \geq n$, a comparison of one integer with another. The refinement does not make that test easier — the test was always easy once the representation was fixed. What the refinement does is move the state to a point where the test passes, and the certificate is the passing test.

Finite confirmation. By Proposition 7.1, $m_{k+1} = 2m_k$ whenever $r_k \neq 0$ (if $r_k = 0$ then $r_{k+1} = 0$, and the valuation is $+\infty$ by convention rather than a large finite number). The ring matters here and we state it: the equality holds in $\mathbb{Z}$, equivalently in $\mathbb{Z}_p$, where ν_p is multiplicative. Under reduction modulo p^n — the setting of the inverses that Section 7 is about — it does *not* hold, because the valuation saturates: once $2m_k \geq n$ the residual is 0 in the quotient and $m_{k+1} = +\infty \neq 2m_k$. What survives reduction, and what the certificate below actually uses, is the inequality

$$m_{k+1} \;\geq\; 2m_k.$$

Two distinct quantities must also be kept apart. Write $\nu_p(r_0)$ for the *actual* initial valuation, and $\underline{m}_0 \leq \nu_p(r_0)$ for a *certified lower bound* on it — the datum an implementation has, as

opposed to the datum a proof has. Iterating the inequality from the certified bound gives $m_k \geq 2^k \underline{m}_0$, so the target $m_k \geq n$ is reached at the latest at

$$K = \min\{k : 2^k \underline{m}_0 \geq n\}.$$

K is a guaranteed deadline, not a hitting time: when $\underline{m}_0 < \nu_p(r_0)$ strictly, the target is reached earlier, and the two coincide exactly when the certified bound is tight. This is an instance of the standard descent of Theorem 5.5 rather than a separate argument: the progress value m_k *grows*, and the paper's distance coordinate is obtained from it by

$$E_n(x) \;=\; \begin{cases} 0, & \nu_p(1-ax) \geq n \quad (\text{including } +\infty), \\ \lceil \log_2(n/\nu_p(1-ax)) \rceil, & 1 \leq \nu_p(1-ax) < n. \end{cases}$$

The two branches are exhaustive on the basin: admissibility forces $\nu_p(1-ax) \geq 1$, so the logarithm is never evaluated at 0, and the confirmed case—including the exact hit $r = 0$—is settled by the first branch rather than by an arithmetic limit. E_n *decreases* by at least one on every *unconfirmed* state, since $m \mapsto 2m$ removes one remaining doubling; no descent is claimed at $E_n = 0$. Multiplicative progress in m is additive descent in E_n, so no extension of Definition 5.1 is required.

Temporal reading. The arithmetic proves the transition law; the temporal layer states its consequences over the trace. With the fluent

$$\mathsf{conf}_n(k) \iff m_k \geq n,$$

Corollary 7.2 gives $\mathbf{F}\,\mathsf{conf}_n$ on every admissible trace, witnessed by a good prefix of length at most K — and $\mathbf{F}\,\mathsf{conf}_n$ lies in $\mathsf{C} \cap \mathsf{L}$: once conf_n has held, every extension satisfies it, and every finite prefix extends to a trace that satisfies it. "At most": K is a guaranteed deadline under a certified lower bound on the initial precision, and the target may be reached earlier; goodness is upward closed, so no minimality is claimed either way.

Proposition 7.3 (Target absorption)**.** *On admissible traces,* $\mathsf{conf}_n(k)$ *implies* $\mathsf{conf}_n(k+1)$.

Proof. If $m_k \geq n$ and $r_k \neq 0$ then $m_{k+1} \geq 2m_k \geq n$, using the inequality form so that the argument is valid in the quotient as well as in $\mathbb{Z}$. If $r_k = 0$ then $r_{k+1} = 0$, and the target holds not vacuously but by the convention $\nu_p(0) = +\infty \geq n$ fixed above. □

Hence $\mathbf{FG}\,\mathsf{conf}_n$ holds here as well. This does *not* reclassify $\mathbf{FG}\,\varphi$, which is $\mathsf{L} \cap \mathsf{K}$ and not finitely monitorable in general; it says that on this restricted class of traces the two formulas have the same extension, because the domain law excludes traces in which confirmation is lost. Evaluating temporal properties against an assumed class of behaviours, and the observation that specifications not monitorable in general may become monitorable under such an assumption, are the subject of assumption-based runtime verification [5, 7]; the present example is an instance of that setting, not a contribution to it.

Scope. Nothing in this section is new arithmetic: the iteration, the residual dynamics, the valuation doubling and the explicit formula are established results [1]. The finite-confirmation step is standard well-founded descent. The section earns its place by exhibiting, on a system where every layer can be checked against independent mathematics, the passage from a recorded trace to a proved law and then to a temporal property — the passage the two examples that follow assume rather than perform. The alignment exploited here — one axis that both advances the state and contracts the residual — is not generic; a contrasting arithmetic boundary case is discussed in Section 8.6.

8. Worked example: primality testing

Primality is the per-input reading of the framework: this section catalogues five established methods that attach a refinement system to the statement "n is prime".

Remark 8.1 (Primality is decidable; what varies is cost). The reading used throughout this section must be stated at the outset, because the natural shorthand for it is misleading. "n is prime" is a *bounded*-quantifier statement: trial division by every $d \leq \lfloor \sqrt{n} \rfloor$ settles it in finitely many steps, two-sidedly, unconditionally, with no hypothesis and no certificate. It is decidable, and it is decidable in polynomial time [18]. Nothing in this paper is needed to make it so, and nothing in this paper should be read as suggesting otherwise.

What is Π_1^0 in form, and refutation-only, is the verdict behaviour of a particular *monitor*: one that enumerates candidate divisors with no a-priori cutoff. Such a monitor emits $\perp$ on finding a divisor and never confirms. When the sections below place the sieve at Floor 4 and describe Miller–GRH or AKS as lifting it to Floor 5, the coordinate being lifted is the cost of confirmation, and the baseline is that unbounded monitor. The lift is a complexity lift, from $\Theta(\sqrt{n})$ to a polynomial in $\log n$; it is not a lift in decidability, monitorability, or arithmetical complexity, and the Π_1^0 label attaches to the monitor's reading rather than to the predicate.

Remark 8.2 (Residual metrics, and the reading they replace). Each refinement system below takes E_k to be the work *remaining*: sieve constraints not yet applied, bases not yet tested, congruences not yet verified, certificate weight not yet discharged. This is forced, not stylistic. The natural alternative — letting E count evidence already accumulated, such as failed tests or divisors found — fails Definition 5.1 on both clauses at once, and an earlier version of this section used it.

Two failure modes, on concrete inputs. With E counting failed Miller tests, $n = 9$ and base $a = 2$ give $E(A_0) = 0$ and $E(A_1) = 1$: a strict *increase* at the first step, so the strict-decrease clause fails immediately. And $n = 7$ has no failing base below $2(\ln 7)^2$, so $E \equiv 0$ from $k = 0$: the target-reach clause then certifies 7 before a single modular exponentiation has been performed. The same pair of failures affects the sieve and AKS readings verbatim. For Pratt, the unweighted leaf count increases at $n = 211$, where $210 = 2 \cdot 3 \cdot 5 \cdot 7$ turns one unproven leaf into three; the weighting of Section 8.4 is what repairs it.

The general point is worth stating once, since it is the most common way a descent argument is misapplied: a Lyapunov metric must measure distance to the target, and accumulated evidence measures distance from the start. The two move in opposite directions.

The five methods solve four different problems. The methods below are often described together in the primality literature, and an earlier version of this section ordered them on a single Floor-N scale. That reading is sound only on the *confirmation coordinate* of a multidimensional classification and misleads on the others, so we begin by making the classification explicit. Five prime-related problems are distinguished here:

- *Decision.* Given n, decide whether n is prime.
- *Certification.* Given n, produce a verifier-checkable certificate that n is prime.
- *Enumeration.* Given a bound N, produce the set $\{p : p \text{ prime}, p \leq N\}$.
- *Existence.* Assert (non-effectively) the existence of a constant, function, or sequence with a stated prime-related property.
- *Construction.* Compute the n-th prime p_n as an effective function of n.

The five methods presented below span exactly the first four; the *construction* of p_n as a closed-form function of n is a long-standing open problem and *is not attempted in this paper* (Section 8.6). Table 4 places each method on five independent coordinates.

Remark 8.3 (Reading of Table 2 restricted to the confirmation coordinate). The Floor-N labels used in the five subsections below are the CONFIRMATION-COORDINATE entries of Table 4, not a total ordering of the methods. In particular, "AKS at Floor 7 above Miller–GRH at

Method	Problem	Assumption	Computation	Certificate	Conf. floor
sieve	Enumeration	unconditional	quasi-linear-in-N	tabulation	4
Miller–Rabin	Decision	probabilistic	rand. poly-in-log n	probabilistic witness	5_ε
Miller–GRH	Decision	GRH	det.-poly-in-log n	verifiable witness	5
AKS	Decision	unconditional	det.-poly-in-log n	verifiable witness	7
Pratt-tree	Certification	unconditional	non-uniform generation	recursive divis. tree	7
Mills	Existence	unconditional	non-effective	none	—

Table 4: Multidimensional classification of the primality methods of Section 8, extended with Miller–Rabin for completeness. "Conf. floor" is the *restricted* Floor-N reading of Table 2 in which Floor-N indexes only the confirmation coordinate (Floor 4 = refutation-only; Floor 5 = two-sided finite in the augmented monitoring problem; Floor 7 = proof-carrying). Assumption strength and computational cost are *independent axes* and do not compose with the floor. Miller–Rabin's entry "5_ε" records that its two-sided verdict is *probabilistic*, sound up to tolerance ε chosen by the augmented monitoring problem—not the unconditional two-sided verdict of Floor 5 proper. Mills's existence result carries no confirmation and therefore has no floor entry. The prime-exponent (Gödel) representation lives on a sixth axis (representation), orthogonal to the five above and outside the scope of this section; it is recorded in Remark 8.6.

Floor 5" means only that AKS's certificate is proof-carrying whereas Miller–GRH's is GRH-conditional—it does not mean AKS is computationally stronger. It is not: Miller–GRH is $\widetilde{O}((\log n)^2)$ per round while AKS is $\widetilde{O}((\log n)^6)$ overall. Assumption strength and computational cost live on independent axes. All floor references in the subsections below should be read under this restriction.

8.1. **Sieve refinement (Enumeration; Floor-4 baseline on the confirmation coordinate).** The Eratosthenes-style sieve already sits at Floor 4.

- $D_k = \mathbb{N} \setminus \bigcup_{i \le k} M_i$ where each M_i is the multiplicity filter of the i-th small prime (residue vector or modular fingerprint);
- $A_0 = \mathbb{N}$ (or the first sieve);
- U adds the next sieve constraint M_{k+1}, exiting with a refutation if $n \in M_{k+1}$;
- $E_k(n) = \max(0,\ \pi(\sqrt{n}) - k)$ counts the sieve constraints *still to be applied*, π the prime-counting function.

E decreases by exactly $c = 1$ at every step at which it is positive, and $E_k = 0$ exactly when every prime up to $\sqrt{n}$ has been tried without a hit, which settles primality. Writing the metric this way rather than as a count of divisors already found is not cosmetic: the latter is non-decreasing in k and equals 0 at $k = 0$ for every prime n, so it satisfies neither clause of Definition 5.1 (see Remark 8.2).

The bound $\pi(\sqrt{n})$ is what makes this terminate, and it is free. A monitor *denied* that bound — one enumerating candidate divisors with no a-priori cutoff — can refute compositeness but never confirm primality, and it is that monitor, not the predicate, which sits at the Floor 4 baseline. The distinction matters for everything that follows and is stated plainly in Remark 8.1.

8.2. **Miller–GRH refinement (Decision; Floor-5, GRH-conditional).** The Miller test under the Generalised Riemann Hypothesis is the canonical Floor 5 lift [17].

- D_k = pairs (n, set of bases tested) with the base set of cardinality at most k;
- $A_0 = (n, \varnothing)$;
- U adds the next base $a \in [2, 2(\ln n)^2]$ and runs the Miller modular-exponentiation test on a, exiting with a refutation on the first base that witnesses compositeness;
- $E_k = \max(0,\ \lceil 2(\ln n)^2 \rceil - k)$ counts the bases *still to be tested*.

Under the Generalised Riemann Hypothesis the bases below $2(\ln n)^2$ suffice: the least witness for a composite n lies in that range, so exhausting the range without a witness settles primality. The explicit constant is Bach's [36]; Miller's own conditional argument [17] does not supply it, and we avoid the phrase "Miller's constant", which invites confusion with Mills's constant in Section 8.5. The logarithm is natural. E decreases by $c = 1$ per step, so at most $O((\ln n)^2)$ updates either drive E to 0 (certified prime) or hit a witness (certified composite).

What this buys is a *cost* lift on the confirmation coordinate — from the $\Theta(\sqrt{n})$ of trial division to $\widetilde{O}((\log n)^3)$ — not a lift in decidability. It moves Floor 4 to Floor 5 for the bounded-cost monitor, which is the sense in which the phrase is used here.

8.3. **AKS deterministic certificate (Decision; Floor-7, unconditional).** AKS [18] produces an unconditional deterministic certificate; the sharpened $\tilde{\mathrm{O}}((\log n)^6)$ bound quoted below is the Lenstra–Pomerance improvement based on Gaussian periods [19], not the original AKS bound (which is $\tilde{\mathrm{O}}((\log n)^{10.5})$ unconditionally, $\tilde{\mathrm{O}}((\log n)^{7.5})$ heuristically).

- D_k = pairs (n, r) with growing modulus r;
- U increases r and verifies the AKS polynomial congruence $(x+a)^n \equiv x^n + a \pmod{x^r - 1,\, n}$ for $a \in [1, \lfloor \sqrt{\varphi(r)} \log n \rfloor]$;
- E_k counts the congruences *still to be verified* at the current modulus r, namely

$$\max(0,\ \lfloor \sqrt{\varphi(r)} \log n \rfloor - k),$$

  with a refutation exit on the first congruence that fails.

$E = 0$ for sufficient r yields a polynomial-time deterministic primality certificate (no GRH assumption). The cost is $\widetilde{O}((\log n)^6)$ but the lift reaches Floor 7 (proof-carrying), since the certificate is a verifiable mathematical object reproducible by any agent in possession of (n, r).

8.4. **Pratt-tree certificate (Certification; Floor-7, generative-only).** The Pratt-tree [20] produces a succinct generative certificate via recursive divisibility witnesses.

- D_k = partial certificate trees with root n and internal nodes for proven prime factorisations of $n - 1$;
- U extends the tree by proving primality of one further small factor of a previously placed internal node;
- $E_k = \sum_{q \text{ unproven leaf}} \lfloor \log_2 q \rfloor$ is the *weighted* count of unproven leaves, with the base cases $q \in \{2, 3\}$ discharged at weight 0.

The weighting is needed, and the floor is not interchangeable with the ceiling. A plain count of unproven leaves can *increase*: expanding the node 211 places the factorisation $210 = 2 \cdot 3 \cdot 5 \cdot 7$ and replaces one unproven leaf by three, so the count goes $1 \mapsto 3$. With the weight $\lceil \log_2 q \rceil$ the measure can stall: at $q = 191$, where $190 = 2 \cdot 5 \cdot 19$, one has $\lceil \log_2 191 \rceil = 8 = \lceil \log_2 5 \rceil + \lceil \log_2 19 \rceil$.

With the floor the descent is uniform, for a reason worth stating. Let $q > 3$ be prime and let $q_1, \ldots, q_m$ be the distinct prime factors of $q - 1$ exceeding 3. Since q is odd, $2 \mid q - 1$, so $\prod_i q_i \le (q - 1)/2$ and therefore

$$\sum_i \lfloor \log_2 q_i \rfloor \ \le\ \sum_i \log_2 q_i \ \le\ \log_2(q - 1) - 1 \ \le\ \log_2 q - 1.$$

The left-hand side is an integer, so it is at most $\lfloor \log_2 q \rfloor - 1$: the uniform quantum is $c = 1$. The factor 2, always present and always free, is what supplies it. Exhaustive check over the first 2000 odd primes: no violation of the decrease, minimum decrement exactly 1. The classification is verified by exhaustive enumeration over the stated range; a uniform proof is left to future work.

Strong convergence at $E = 0$ yields a Pratt certificate of size $O(\log^2 n)$ and verification in $\widetilde{O}((\log n)^3)$. Certificate *construction* is a separate matter: it requires the complete factorisation of $n - 1$ at every node, for which no polynomial-time algorithm is known (Remark 3.7). Floor 7 reached, generative variant, at that construction cost.

8.5. **Mills's constant (Existence; not a genuine confirmation-floor lift).** Mills's theorem [21] asserts the existence of $A > 1$ such that $\lfloor A^{3^n} \rfloor$ is prime for every $n \geq 1$. Phrased as a Hensel–Newton tuple:

- D_k = approximations $A_k \in \mathbb{R}$ satisfying $\lfloor A_k^{3^i} \rfloor = p_i$ for $i \leq k$;
- U refines $A_k \to A_{k+1}$ by inverting the next constraint;
- E counts primes p_i misidentified by the current approximation.

Convergence requires the primes p_i to be supplied in advance; the refinement is parasitic on the very object it is supposed to certify. This is a methodologically important boundary case: the (D, A_0, U, E) shape is present but the structural-zero guarantee is circular (the primes are inputs, not outputs of the system). Mills lies *outside* the Floor-N hierarchy as a refinement that does not satisfy Definition 3.4.

Remark 8.4 (Why five instances). The five instances span the Floor-N lift hierarchy: sieve at Floor 4 (baseline), Miller at Floor 5 (conditional lift), AKS and Pratt at Floor 7 (proof-carrying), Mills as boundary. Each illustrates a different structural-zero guarantee: Miller relies on the multiplicative group structure of $(\mathbb{Z}/n\mathbb{Z})^\times$ under GRH; AKS on cyclotomic polynomial congruences; Pratt on recursive divisibility witnesses; Mills on advance knowledge of the prime sequence (and is therefore not a genuine lift).

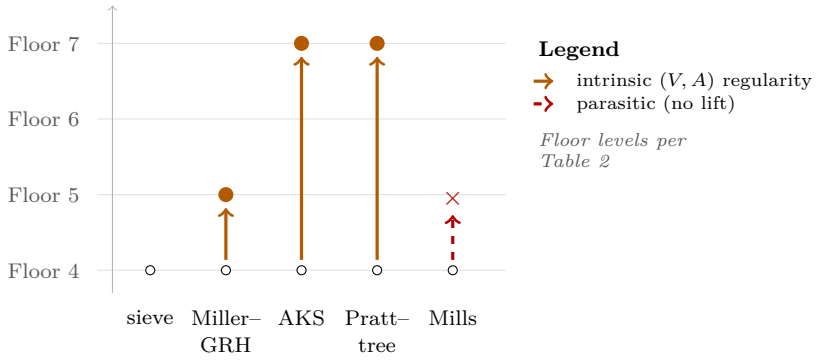


Figure 1: Confirmation-coordinate reading of the five primality refinement systems of Section 8. *The figure plots ONLY the confirmation-coordinate row of Table 4*—not a total ordering across assumption strength, computational cost, or certificate generation cost. Sieve (column 1) sits at the Floor 4 baseline (Π_1^0, refutation-only). Miller–GRH (column 2) lifts to Floor 5 ($\mathsf{S} \cap \mathsf{C}$) conditionally on the Generalised Riemann Hypothesis (assumption axis of Table 4). AKS (column 3) and Pratt-tree (column 4) both reach Floor 7 (proof-carrying); the certificate types differ (deterministic congruence check vs. generative tree witness). Mills (column 5) is the boundary case: parasitic convergence, no confirmation floor. On the computational-cost axis (not shown), AKS is $\widetilde{O}((\log n)^6)$ and Miller–GRH is $\widetilde{O}((\log n)^2)$ per round, so vertical ordering here is NOT an efficiency ordering. All four genuine lifts shown use intrinsic (V, A) regularity (Section 6.1); extrinsic and active-tuning regularity become relevant in the parametric-monitor case of Section 9.

8.6. **Two problems this section does not attempt.** Two prime-related problems fall outside the scope of the five methods above and are named here explicitly so that they are not implicitly claimed.

- *Construction of p_n as an effective function of n.* No *efficiently computable* closed form is known that returns the n-th prime. Three things are easily conflated here, and the taxonomy needs all three. *(a) Direct but parasitic.* Formulae computing p_n directly from the index, such as Mills's [21], presuppose a constant that already encodes the primes (Section 8.5). *(b) Direct, effective, infeasible.* Wilson-theorem formulae, of which Willans's [37]

is the standard representative, are direct functions of n, presuppose no pre-supplied constant, and are perfectly effective; they are merely astronomically impractical, since evaluating them requires factorials of the magnitude of the answer. Their existence is what forbids reading "direct formula" as "parasitic" and is why the contrast in (a) is not the whole story. *(c) Effective recurrences.* Exact *recurrences* computing p_{n+1} from $p_1, \ldots, p_n$ do exist: Gandhi's primorial construction, derived and generalised by Golomb [24, 25], together with the recursion of Keller [26], which expresses p_{n+1} through a limit as an auxiliary parameter s tends to infinity and is therefore not constructive as stated; Cloitre [27] makes it effective by proving that a *finite* $s \leq p_n$ suffices when combined with a ceiling, for every $n \geq 1$. These are answer a different question: the refinement systems above solve DECISION or CERTIFICATION on a per-input n, whereas enumeration of p_n as a function of n is not addressed by this paper.

- *Factorisation of composite n from a primality test.* A DECISION verdict of "composite" from AKS (Section 8.3) or a Miller-type test (Section 8.2) does not, in general, produce the factorisation of n. Some probabilistic primality tests can occasionally return a nontrivial factor when the witness's exponent structure exposes one, but this is not guaranteed and is orthogonal to the finite-confirmation mechanism at issue here.

Remark 8.5 (Filtering and leading extraction act on different axes). The contrast with Section 7 locates what made that example close. Let $P_n = \prod_{j \leq n} p_j$ and let $G_n(z) = \sum_{\gcd(k, P_n)=1} z^k$ collect the survivors of the corresponding coprimality filter, the term $k = 1$ included. Writing $(\mathcal{S}_p f)(z) = f(z^p)$, the next stage satisfies $G_{n+1} = (I - \mathcal{S}_{p_{n+1}})G_n$ exactly, while the next prime is $p_{n+1} = \nu_z(G_n - z)$, where ν_z denotes the lowest exponent carrying a nonzero coefficient — not the p-adic ν_p of Section 7. With $p = p_{n+1}$,

$$G_{n+1}(z) - z \;=\; (G_n(z) - z) - G_n(z^p),$$

where both summands have valuation p and leading coefficient 1: their leading terms cancel, which is precisely the removal of p_{n+1} from the survivors. The valuation of the updated series is therefore the next query itself, rather than a datum recoverable uniformly from any fixed finite window of leading coefficients, since prime gaps are unbounded. In Section 7 a single axis both advances the state and contracts the residual; here stage evolution and leading-term extraction are distinct, the analytic branch isolating the survivor through a separate parameter. Whether some enriched valuation couples the two axes is open. The filtering identity itself is established [24, 25]; only the comparison is at issue.

Remark 8.6 (Prime-exponent ("prime-base") representation). An adjacent research line proposes identifying $n \in \mathbb{N}^+$ with its prime-exponent vector $(e_2, e_3, e_5, \ldots)$ of *finite support* (finitely many nonzero entries) satisfying $n = 2^{e_2} \cdot 3^{e_3} \cdot 5^{e_5} \cdot \cdots$. This is the canonical Gödel encoding of the fundamental theorem of arithmetic and realises the isomorphism $(\mathbb{N}^+, \times) \cong \bigoplus_{p \text{ prime}} (\mathbb{N}, +)$ as a bijection between $\mathbb{N}^+$ and the finitely supported $\mathbb{N}$-valued functions on the set of primes. Multiplication of integers corresponds to coordinate-wise addition of exponent vectors; ordinary integer addition has no closed-form counterpart on the exponent side.

Read against the classification of Table 4, the representation is *indexically parasitic on the primes*: the digit positions are labelled by primes, so the encoding cannot describe the labelling scheme it presupposes. This is a scope condition, not a defect: the representation is a legitimate encoding, not a failed computation. It differs from Mills's parasitism (Section 8.5), which is *computationally* circular—Mills's constant A is defined in terms of an already chosen prime sequence, so Mills claims a generative reach it cannot deliver. The prime-base representation claims no generative reach and therefore fails no generative claim. Both presuppose the primes; only Mills promises to produce them. The representation occupies a sixth axis of the classification—*representation*—that is orthogonal to the five above and outside the scope of the present development.

9. Worked example: parametric monitor enforcement

This section instantiates Definition 6.5 with the augmented-monitor parametric family of [10]. Pipeline anchor: the real-valued signal Q below is at Stage 0 of the companion pipeline; the augmented monitor M with parameter point $(\theta_{\mathrm{up}}, \theta_{\mathrm{down}}, \pi) \in \Pi$ is at Stage 1; the verdict-trace events $V = \{\mathsf{V}, \mathsf{I}, \varepsilon\}$ are at Stage 2; the emitted ω-trace $E_M(Q)$ is at Stage 3; the target LTL formula φ is the Stage 4 trace property. The active-tuning regularity is constructed by the refinement system through the adaptation rule U_φ: each adaptation step updates the Stage 1 parameter point in response to the Stage 3 trace prefix, and the Lyapunov metric E evaluates the Stage 3 prefix against the satisfaction set of φ at Stage 4.

In the tuning-power hierarchy $L_1 \subsetneq L_2 \subsetneq L_3$ of the companion [10], this enforcer sits at the *closed-form corner*: the structural-zero guarantee certifies φ on the infinite trace from the original stream's intrinsic regularity, with neither an online parameter search nor signal modification (L_3). The general effort-optimal selection across the full hierarchy is left to future work.

Let M be the augmented monitor of the companion work [10]: $M(\theta_{\mathrm{up}}, \theta_{\mathrm{down}}, \pi)$ acts on a real-valued signal $Q \in \mathbb{R}^{\mathbb{N}}$ and emits a verdict trace $E_M(Q) \in V^\omega$ over the ternary event alphabet $V = \{\mathsf{V}, \mathsf{I}, \varepsilon\}$. The synthesis algorithm of that paper takes a *finite* target event pattern E^* and emits a static tuning-parameter assignment that produces E^* on the given signal in time $O(n + T)$. A strict generalisation lifts this from one-shot synthesis on a finite specification to *online adaptation* against an infinite specification: replace E^* by an LTL formula φ on the verdict trace $E_M(Q) \in V^\omega$, and let the tuning parameters evolve as a function of the current parameter point and the trace prefix emitted so far. The monitor ceases to be a passive observer and becomes an active controller: its parameters are continually adjusted to drive the verdict trace toward satisfaction of φ.

Definition 9.1 (Enforcement via parametric adaptation)**.** Let φ be a property on V^ω in the CLASP partition, let Π denote the parameter space $\Pi := \{(\theta_{\mathrm{up}}, \theta_{\mathrm{down}}, \pi) : \theta_{\mathrm{up}}, \theta_{\mathrm{down}} \in (\mathbb{R} \to V)^{\mathbb{N}}, \pi \in \mathcal{C}\}$ of the augmented monitor (where $\mathcal{C}$ is the convention algebra of [10]), and let $\mathrm{state}_n \in \Pi$ be the parameter point at step n. A *parametric-adaptation enforcer for* φ is a computable map

$$U_\varphi : \Pi \times V^* \to \Pi$$

such that, given the current parameter point state_n and the verdict-trace prefix $E_M(\sigma \restriction_n) \in V^*$ emitted by the augmented monitor under the previous parameter trajectory, U_φ returns the next parameter point state_{n+1}. The enforcer is *convergent for* φ *on* Q if there exists $k \in \mathbb{N}$ such that, for all $n \geq k$, the trace prefix $E_M(\sigma \restriction_n)$ produced under the sequence of parameter points $(\mathrm{state}_m)_{m \leq n}$ satisfies the bounded-prefix witness of φ (e.g. the constant-v tail required by $\mathbf{FG}(b = v)$, or the alternation pattern required by $\mathbf{GF}(b = v)$).

Remark 9.2 (Enforcement via adaptation versus enforcement via edits)*.* The classical runtime-enforcement framework of Schneider's security automata and the safety/co-safety enforcement constructions of Falcone, Fernandez and Mounier [11] treat enforcement as an *event-stream editing* operation: the enforcer suppresses, inserts, or buffers events on the monitor output to coerce compliance with a target property. The mechanism of Definition 9.1 is structurally different: the verdict trace is left untouched, and adaptation acts on the *parameter space* of the monitor itself. The augmented monitor's parametric algebra (Π above) is what makes this mechanism available; absent parametric structure, only edit-based enforcement is possible.

Remark 9.3 (Enforcement as a Hensel–Newton refinement)*.* The parametric-adaptation enforcer of Definition 9.1 instantiates the (D, A_0, U, E) refinement tuple of Section 3: $D_k := \Pi$ at step k, A_0 the coarse initial parameter point, U the adaptation rule, and E a Lyapunov-style distance between the current verdict trace and the satisfaction set of φ. When the augmented monitor's monotonicity in each parameter coordinate holds, the mapping instantiates the active-tuning regularity source of Section 6; convergence $E \to 0$ in finite time then reduces to the general statement of Theorem 5.5. This remark is a naming convention, not a new mathematical

claim—the enforcer's convergence is what Theorem 5.5 says once the tuple identification is made explicit.

Remark 9.4 (Non-degeneracy assumptions required for the enforcer). Definition 9.1's convergence clause is licensed by Theorem 5.5 only when three non-degeneracy assumptions hold on the pair (φ, U_φ):

(ND1) *Monotonicity of E in U_φ*: for every $\text{state}_n \in \Pi$ and every verdict prefix $\sigma \in V^*$, either E is already zero at state_n or the update $\text{state}_{n+1} := U_\varphi(\text{state}_n, \sigma)$ satisfies $E(\text{state}_{n+1}) \leq E(\text{state}_n) - c$ for a fixed strict-decrease constant $c > 0$;

(ND2) *Non-triviality of φ on the parameter orbit*: the satisfaction set of φ is not reached by every parameter point in Π (otherwise "enforcement" is vacuous and any U_φ would qualify);

(ND3) *Cooperative environment*: the signal Q is fixed throughout the adaptation, and the observed verdict prefix is the one M actually emits under the parameter trajectory $(\text{state}_m)_{m \leq n}$ produced by U_φ. An adversarial environment that modifies Q in response to the parameter trajectory is outside the scope of the present convergence statement.

We state plainly where this leaves the mechanism. In the parametric-monitor instance of Section 9, (ND1)–(ND3) are *open* conditions on the target formula φ and the adaptation rule U_φ, and this paper exhibits no instance in which all three are discharged. Definition 9.1 and the contrast of Table 5 are therefore offered as a *distinction and an open direction*, not as a validated mechanism: what is established is that a monitor whose configuration is exposed as a computable parameter space admits an enforcement notion that does not rewrite the verdict stream, and what is not established is that any non-degenerate such enforcer exists. Supplying one, and settling non-degeneracy in general, is future work (Section 12 outlook).

10. Edit-based versus adaptation-based enforcement

We collect in Table 5 the structural differences between the classical edit-based enforcement mechanism of [16, 11] and the adaptation-based mechanism of Definition 9.1. The differences are five-fold: the object modified, whether the output is left untouched, whether a parametric monitor is required, the type of convergence guarantee, and the underlying framework.

Aspect	Edit-based enforcement	Adaptation-based enforcement
What is modified	Verdict trace (events)	Monitor parameters
Output untouched?	No (edited)	Yes (only generation rule changes)
Requires parametric monitor?	No	Yes
Convergence guarantee	Per-event decisions	Lyapunov refinement
Underlying framework	Edit-functions / safety automata	Hensel–Newton refinement-tuple
Pipeline locus	Stage 3 endomorphism	Stage 1 refinement

Table 5: Structural comparison of edit-based enforcement (Schneider security automata [16]; Falcone–Fernandez–Mounier [11]) and adaptation-based enforcement (Definition 9.1). The two mechanisms are complementary, not in conflict. When a parametric monitor is available ([10]), adaptation-based enforcement is strictly less intrusive than edit-based: the verdict stream is never overwritten or fabricated; only the generation rule is adjusted.

Remark 10.1 (Pipeline-orthogonality of edit-based enforcement). The third row of Table 5 records the reason edit-based enforcement cannot be re-described as adaptation: a monitor that does not expose its configuration as a parameter space supports only edit-functions on the trace. Adaptation becomes available exactly when the configuration is separated from event generation, which is the first of the two structural differences noted in Section 11.7. We state this as an observation about the two mechanisms, not as a theorem.

11. Related work and connections

11.1. **Companion work (internal).** The Hensel–Newton refinement tuple is dual to the universal attribution theorem of the companion work [8]: the theorem says that every monitor canonically yields an $\mathsf{L} \cap \mathsf{K}$ pullback (*internal attribution*); the refinement-tuple says that external supporting structure promotes refutation-only properties to finite-time decidability (*constructive attribution*). The augmented monitor synthesis algorithm of [10] is the canonical event-generator instance of active-tuning regularity, as expanded in Section 9.

11.2. **External pipeline comparison.** We are aware of no prior framework that exposes the same six-stage decomposition. The closest analogues, ordered by pipeline coverage, are:

- *Bauer–Leucker–Schallhart* LTL_3 [12]. The standard monitor-synthesis pipeline of three-valued runtime verification: *specification* → *monitor synthesis* → *observation* → *verdict* $\{\top, \bot, ?\}$. The signal and event stages are conflated into "observation", the monitor configuration is fused with monitor execution, and no meta-property layer exists. Mapping to our pipeline: their pipeline covers our Stage 3→Stage 4 boundary; Stage 0–2 are implicit, Stage 5 is absent.
- *Maler–Ničković STL* [13]. Continuous-signal monitoring with quantitative robustness ρ: *signal* → *robustness evaluation* → *Boolean verdict*. Has an explicit signal stage (Stage 0), but the event-generation step is folded into ρ-computation, and there is no meta-property layer. Mapping: covers our Stage 0→Stage 4 in three stages.
- *Falcone–Fernandez–Mounier enforcement* [11]*; Schneider security automata.* Edit-based enforcement: *verdict trace* → *enforcer/editor* → *compliant trace*. Operates downstream of our Stage 3 and modifies the trace itself rather than the monitor configuration; our Remark 9.2 contrasts this with parametric adaptation. No analogue of Stage 5 or of the bootstrap.
- *Aceto–Achilleos–Francalanza et al. monitorability hierarchies* [14]. Spectrum of monitorability fragments of Hennessy–Milner logic with recursion, characterising the trade-off between monitoring guarantees and property expressivity. Closest in spirit to our CLASP-cell hosting at Stage 4, but stays at the property layer; does not develop signal/monitor decomposition.
- *Bonakdarpour–Finkbeiner HyperLTL monitoring* [15]. Tree-shaped Kripke structures grown at runtime to model-check hyperproperties. This is the prior framework closest to our bootstrap construction $\tau_k = s_{k+1}$: both promote a trace at one layer to an input of a higher-order monitoring step. The bootstrap of Section 2.4 is the linear-time, alphabet-constrained dual of the branching-time Kripke-tree construction.

Remark 11.1 (What is novel about the six-stage pipeline). The above comparison shows that prior frameworks expose at most a three-to-four stage pipeline and do not separate the monitor configuration μ (Stage 1) from the event-generation map ε_μ (Stage 2), nor the meta-property Ψ (Stage 5) from the trace property Φ (Stage 4). The six-stage decomposition of the companion work [8] is, to our knowledge, the first to (i) name all five intermediate objects explicitly, (ii) place the meta-property as *logically prior* to monitor configuration, and (iii) carry a cross-axis alphabet constraint $T3_{k+1} = D1_k = V_{\mu_k}$ between layers of recursion. The Hensel–Newton refinement tuple relies on this decomposition: its components occupy specific loci on the pipeline (Conjecture 2.1), and the three regularity sources separate cleanly only after the Stage 1 / Stage 5 split is made (Section 2.3).

11.3. **Classical foundations, and what is inherited.** The machinery this paper uses to state its results is standard, and saying so precisely is a precondition for stating what is not.

The property taxonomy. The safety/liveness decomposition is Alpern and Schneider's [31]; the names *recurrence* for $\mathbf{GF}\,\varphi$ and *persistence* for $\mathbf{FG}\,\varphi$, and the hierarchy they sit in, are Manna and Pnueli's [32]; the good-prefix and bad-prefix vocabulary is Kupferman and Vardi's [33]; and the refinement of the classification according to what a monitor can actually decide is Peled and Havelund's [9], whose nine-cell verdict-axis partition is the external reference point for the sixteen-cell partition of [8] used here (Section 1). This paper contributes nothing to that body of work. It consumes it.

The descent argument. Theorem 5.5 is a well-founded descent with a uniform decrement: a non-negative measure that drops by at least $c > 0$ at every non-target step reaches the target in boundedly many steps. That primitive is Floyd's [34], and its modern automated form — the synthesis of ranking functions, and the transition-invariant machinery that generalises them — is due to Podelski and Rybalchenko [35]. We claim no novelty for it, and Remark 5.11 already says where the difficulty actually lies: not in the descent, which is three lines, but in constructing an E whose orbit values avoid $(0, c)$. What is specific here is the packaging — the four-component tuple, the target-reach clause relativised to a declared trace class, and the separation of the finite hitting statement from the temporal one — not the primitive.

11.4. **Abstraction refinement, and what is specific here.** The architecture of this paper — represent a problem in a space where the target becomes a simple test, refine that representation until the test succeeds — is not new, and it belongs to the same field as this paper rather than to a distant one. Saying so precisely is what allows the narrower claim that follows.

Abstract interpretation [28] computes program properties in an abstract domain ordered by precision, related to the concrete domain by abstraction and concretization maps, with the property obtained as an approximated fixed point and transported back by a soundness theorem. Predicate abstraction [29] is the case in which the abstract domain is the Boolean space of valuations of a finite predicate family, so that a possibly infinite concrete state space is represented by finitely many abstract states. Counterexample-guided abstraction refinement [30] closes the loop: when the abstraction admits a counterexample with no concrete counterpart, the abstraction is refined — equivalence classes are split — and the check is repeated, until either a genuine counterexample refutes the property or its absence establishes it.

Everything in the previous paragraph is inherited, not contributed. The refinement system of Definition 3.1 is a refinement loop of that kind, with the monitor configuration in the role of the representation and the verdict in the role of the check. We claim nothing for the pattern itself.

What is specific is the *shape of the bound*, and it is worth stating exactly because it is narrow. Counterexample-guided refinement is not guaranteed to terminate on infinite-state systems, and implementations carry stopping criteria for that reason; where a bound does exist it is typically a *cardinality* bound, read off the finiteness of the abstract space — at most 2^m valuations for m predicates, or a bound in the number of equivalence classes. Theorem 5.5 bounds the refinement differently: by $\lceil E(A_0)/c \rceil$, a quantity derived from a metric on the configurations and from the initial error of the particular instance. It does not require the representation space to be finite, and it varies with the instance rather than with the size of the space. The two kinds of bound are not comparable as strengths — ours needs a uniform progress quantum, which is a real hypothesis and often the hard one to supply (Remark 5.11) — but they are obtained from different data, and that difference is the contribution in this direction.

11.5. **Assumption-based runtime verification.** This is the closest prior art to the paper's central move, and it deserves a direct comparison rather than a passing citation.

In assumption-based runtime verification (ABRV) a monitor is synthesised not from the specification alone but from the specification together with a model of the system, which encodes the assumptions the input traces are expected to satisfy; specifications that are not monitorable

in general may become monitorable under such an assumption [5, 7]. The following comparison is what matters here.

	ABRV	this paper
what restricts the futures	a system model A	an admitted trace class $\mathcal{T}$ (Definition 3.2)
prefix notion	safety / co-safety *relative to* A	good prefix relative to $\mathcal{T}$
where the assumption comes from	given as a model	catalogued by source (Section 6)
what is added on top	monitor synthesis, partial observability, resets	a progress measure E and a step bound

The first two rows are the important ones, and they are not a contrast. ABRV already has the notion of a property being safe or co-safe *relative to* an assumption, and monitorability relative to it; the relative good prefix of Definition 3.2 is that same idea stated at the level of prefixes. We therefore do not present it as new. Its role here is bookkeeping, and the point is worth stating once because it is easy to lose: a certificate quantified over an admitted class does not transport to a changed class. If the system is intervened upon — a parameter moved, a controller shielded — the admitted class changes with it, and a confirmation proved under the old class says nothing under the new one. That is elementary assumption-relative reasoning rather than a result, but writing the quantifier explicitly is what makes it visible, and what stops the generic theorem from being read as proving more than it does. The contribution in this direction, if any, is the *bound*: ABRV answers whether a verdict is reachable under an assumption, and Theorem 5.5 answers in how many steps, given a progress measure. Whether that bound is itself already available in the ABRV setting we have not established, and we do not claim it is not.

11.6. **Enforcement: what is and is not settled.** Definition 9.1 moves a parameter of the observed system rather than editing the observed trace, and Table 5 contrasts that with the edit automata of Schneider [16] and Falcone, Fernandez and Mounier [11]. Against those two, the distinction is real.

Against a wider literature it is not established, and we say so rather than leaving the reader to discover it. Shield synthesis [6] and the supervisory-control reading of shielding also act on the *system* rather than on the recorded trace: a shield disables or overrides actions of the controller it supervises, which is an intervention on the generator, not a rewriting of an observation. Where the parameter being moved is a parameter of the generator rather than of the monitor, the distinction drawn in Section 10 narrows further. A targeted comparison against shielding, supervisory control, runtime assurance and the self-adaptive-systems literature has not been carried out here. Until it is, the status of the adaptation/edit distinction is *unresolved* beyond the two frameworks it is drawn against, and nothing in this paper should be read as claiming more.

11.7. **What the comparison establishes, and in what form.** An earlier version of this section cast the comparison of Section 11.2 in categorical language, treating each framework as a small category and asking for functors into the six-stage pipeline. That apparatus has been removed, because it was not carrying theorem-strength and two of its claims were false as stated: no class of the form "no functor exists" can be non-empty, since a constant functor into any non-empty category always exists, and no category lacks an identity morphism on any of its objects. A third notion — faithfulness "on more than three of the six objects" — conflated faithfulness, which is a property of the action on hom-sets, with injectivity on objects. The source categories were also never constructed beyond an ad hoc object list read off other authors' prose, which makes a negative existence claim over them unfalsifiable as posed.

The substance the comparison was meant to carry survives without it, and is already stated in Section 11.2. Two structural features distinguish the six-stage decomposition from the five frameworks reviewed there. The first is the explicit separation of the monitor configuration

from event generation: it is exactly this separation that lets an enforcement move act on the configuration rather than on the trace, which is what Definition 9.1 requires in order to be statable at all. The second is that the meta-property is placed as logically prior to the configuration rather than derived from it. We record these as differences in decomposition, not as a proved minimality result: no claim is made that the six-stage pipeline is the smallest such decomposition, and we have not attempted to prove one.

11.8. **A geometric reading: embedding stability.** The two structural primitives of Section 3 admit a reading in geometric terms:

- The Lyapunov error metric E is an *embedding-stability function* on the orbit $\{A_k\}_{k\in\mathbb{N}}$ of monitor configurations; strict-decrease (Definition 5.1) is the discrete analogue of asymptotic stability for the embedding of the verdict trace into the satisfaction set of Φ.
- The structural-zero guarantee (Definition 5.4) is the discrete analogue of an *attractor condition*: no orbit can stall at any $E > 0$ level, so the only stable fixed point is $E = 0$, the target embedding.

This is the level at which the geometric reading is developed here; deeper formalisation (e.g. via differential inequality bounds on E or via the use of the embedding as a Lyapunov candidate in a dynamical-systems sense) is left to future work.

Dual to internal attribution. The companion theorem of [8] on universal attribution describes *internal* attribution: every monitor $M\colon \Sigma^* \to V$ with $|V| \geq 2$ yields a canonical $\mathsf{L} \cap \mathsf{K}$ pullback property on Σ^ω via the continuous extension $E_M\colon \Sigma^\omega \to V^\omega$. The refinement tuple of the present paper realises the dual: *external* attribution, by which a Π^0_1 property is promoted to $\mathsf{S} \cap \mathsf{C}$ via supplied structured evidence. Together the two strands span the operational reach of Layer-N monitorability: the theorem says what $\mathsf{L} \cap \mathsf{K}$ a single monitor implicitly carries; the tuple says what supporting structure lifts a refutation-only Π^0_1 property sideways into $\mathsf{S} \cap \mathsf{C}$.

12. Discussion and outlook

12.1. **What the framework buys.** The Hensel–Newton refinement tuple supplies a constructive *external* mechanism for finite-time confirmation of infinitary trace-property claims that the CLASP partition of [8] classifies as refutation-only or non-monitorable. Crucially, the lift does not alter the intrinsic CLASP cell of the target property Φ; it provides a separable certificate that operationally reaches a finite verdict. This separability is what makes the mechanism compositional: a property classified at Floor 4 remains Floor 4 in the CLASP sense even after the Floor-N lift, and the certificate can be archived, transmitted, or independently re-verified.

12.2. **Limits of the present development.** Three classes of limit are visible at this draft state.

- *Computability ceiling.* The framework presumes total recursive U and E. An interesting open question is whether the framework admits a relaxation under which U is only partially recursive but the structural-zero guarantee nevertheless ensures decidable progress in finite expected time.
- *Hilbert-style certificates.* Floors ≥ 7 transition from refinement evidence to formal certificate generators. The boundary between Floor 6 (strategic enforceability) and Floor 7 (proof-carrying closure) is essentially the boundary between *operational* and *deductive* certificates; a precise calibration of this boundary requires interfacing with the AI-epistemology companion work.

- *Adversarial regularity.* Active-tuning regularity (Section 6.3) presumes the refinement system controls the monitor's parameters in a cooperative environment. An adversarial environment may contest the parameter updates; characterising the regime in which adaptation-based enforcement remains effective under adversarial signal generation is a natural extension.

12.3. **Outlook.** The natural follow-on directions are:

- formal development of the Lyapunov-as-embedding-stability reading of Section 11.8 into a dedicated treatment of geometric stability;
- extension of the primality worked examples (Section 8) to non-number-theoretic instances (program verification, hybrid systems, control synthesis) that exhibit intrinsic regularity beyond the number-theoretic;
- an epistemic-calculus reading in which the refinement certificate plays the role of Popperian corroboration;
- a companion treatment reading the Floor 6 lift as the constructive face of positive-AI monitoring;
- a structure-aware-SAT remark, where Miller–Rabin and AKS-style filters retrospectively instantiate the refinement-tuple framework;
- a future worked example reading embedding-stability refinement via Lyapunov-style metrics on perturbation traces.

13. Conclusion

We have introduced the Hensel–Newton refinement-tuple framework (D, A_0, U, E) as a structured external mechanism for the finite-time confirmation of properties on monitor verdict traces that the CLASP partition of [8] classifies as refutation-only (Π_1^0 in the naive monitor's reading) or non-monitorable. The non-monitorable targets treated here occupy the cell $\mathsf{L} \cap \mathsf{K}$ and split across two hierarchy levels: recurrence $\mathbf{GF}\,\varphi$, whose quantifier pattern $\forall i\, \exists j \geq i$ places it at Π_2^0, and persistence $\mathbf{FG}\,\varphi$, whose pattern $\exists i\, \forall j \geq i$ places it at Σ_2^0. The arithmetical level and the CLASP cell are different coordinates: both schemata share the cell and differ in the level. The framework is grounded in the six-stage signal–monitor–events–trace–property–meta pipeline of the companion work and occupies specific loci on that pipeline (Conjecture 2.1): the refinement domain and initial approximation at Stage 1, the update rule on the Stage 1→Stage 3→Stage 1 edge, the Lyapunov error metric on the Stage 3→Stage 4 edge.

The convergence theorem (Theorem 5.5) supplies a violation bound of $\lceil E_{\max}/c \rceil$ on Lyapunov refinement systems with strict-decrease constant c and structural-zero guarantee, and catalogues three regularity sources (Section 6) supplying that guarantee: intrinsic algebraic regularity of (V, A), extrinsic regularity inherited from the generating process, and active-tuning regularity constructed runtime via parametric adaptation. Five worked examples from primality testing (Section 8) illustrate the Floor-N hierarchy.

The framework is dual to the universal attribution theorem of [8]. Two features of the decomposition distinguish it from the prior runtime-verification frameworks reviewed in Section 11.2: the explicit separation of monitor configuration from event generation, and the placement of the meta-property as logically prior to the configuration (Section 11.7). These are differences in decomposition; no minimality or non-embeddability result is claimed.

References

[1] J.-G. Dumas, *On Newton–Raphson iteration for multiplicative inverses modulo prime powers*, arXiv:1209.6626, 2012.

[2] J. von zur Gathen, *Hensel and Newton methods in valuation rings*, Mathematics of Computation **42**(166) (1984) 637–661. doi:10.1090/S0025-5718-1984-0736459-9.

[3] M. Avendaño, T. Krick and A. Pacetti, *Newton–Hensel Interpolation Lifting*, Foundations of Computational Mathematics **6**(1) (2006) 82–120. doi:10.1007/s10208-005-0172-3.
[4] J. Esparza and V. Fischer, *Runtime Verification for LTL in Stochastic Systems*, arXiv:2508.07963 (2025).
[5] A. Cimatti, C. Tian and S. Tonetta, *Assumption-based Runtime Verification*, Formal Methods in System Design **60**(2) (2022) 277–324. doi:10.1007/s10703-023-00416-z.
[6] R. Bloem, B. Könighofer, R. Könighofer and C. Wang, *Shield Synthesis: Runtime Enforcement for Reactive Systems*, in: Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2015), Lecture Notes in Computer Science **9035**, Springer, 2015, pp. 533–548. doi:10.1007/978-3-662-46681-0_51.
[7] T. A. Henzinger and N. E. Saraç, *Monitorability Under Assumptions*, Runtime Verification (RV 2020), Lecture Notes in Computer Science, Springer, 2020. doi:10.1007/978-3-030-60508-7_1.
[8] D. Bragetti, *The Intersection Algebra of Safety, Cosafety, Liveness, and Coliveness over Linear Temporal Logic*, Zenodo, version v2, 1 July 2026. doi:10.5281/zenodo.20155195.
[9] D. Peled and K. Havelund, *Refining the Safety–Liveness Classification of Temporal Properties According to Monitorability*, in: Models, Mindsets, Meta: The What, the How, and the Why Not?, Lecture Notes in Computer Science **11200**, Springer, 2019, pp. 218–234. doi:10.1007/978-3-030-22348-9_14.
[10] D. Bragetti, *Touch and Crossing: Dual Primitive Modalities of Threshold Monitors*, Zenodo, 2026. doi:10.5281/zenodo.21078483.
[11] Y. Falcone, J.-C. Fernandez and L. Mounier, *What Can You Verify and Enforce at Runtime?*, International Journal on Software Tools for Technology Transfer **14**(3) (2012) 349–382. doi:10.1007/s10009-011-0196-8.
[12] A. Bauer, M. Leucker and C. Schallhart, *Runtime Verification for LTL and TLTL*, ACM Transactions on Software Engineering and Methodology **20**(4) (2011) Article 14, 64 pp. doi:10.1145/2000799.2000800.
[13] O. Maler and D. Ničković, *Monitoring Temporal Properties of Continuous Signals*, In: *Formal Techniques, Modelling and Analysis of Timed and Fault-Tolerant Systems (FORMATS/FTRTFT 2004)*, Lecture Notes in Computer Science **3253**, Springer, 2004, pp. 152–166. doi:10.1007/978-3-540-30206-3_12.
[14] L. Aceto, A. Achilleos, A. Francalanza, A. Ingólfsdóttir and K. Lehtinen, *Adventures in Monitorability: From Branching to Linear Time and Back Again*, Proceedings of the ACM on Programming Languages **3**(POPL) (2019) Article 52, 29 pp. doi:10.1145/3290365.
[15] B. Bonakdarpour and B. Finkbeiner, *Runtime Verification for HyperLTL*, In: *Runtime Verification (RV 2016)*, LNCS **10012**, Springer, 2016, pp. 41–45. doi:10.1007/978-3-319-46982-9_4.
[16] F. B. Schneider, *Enforceable Security Policies*, ACM Transactions on Information and System Security **3**(1) (2000) 30–50. doi:10.1145/353323.353382.
[17] G. L. Miller, *Riemann's Hypothesis and Tests for Primality*, Journal of Computer and System Sciences **13**(3) (1976) 300–317. doi:10.1016/S0022-0000(76)80043-8.
[18] M. Agrawal, N. Kayal and N. Saxena, *PRIMES is in P*, Annals of Mathematics **160**(2) (2004) 781–793. doi:10.4007/annals.2004.160.781.
[19] H. W. Lenstra Jr. and C. Pomerance, *Primality testing with Gaussian periods*, Journal of the European Mathematical Society **21**(4) (2019) 1229–1269. doi:10.4171/JEMS/861.
[20] V. R. Pratt, *Every Prime has a Succinct Certificate*, SIAM Journal on Computing **4**(3) (1975) 214–220. doi:10.1137/0204018.
[21] W. H. Mills, *A Prime-Representing Function*, Bulletin of the American Mathematical Society **53**(6) (1947) 604. doi:10.1090/S0002-9904-1947-08849-2.
[22] H. Poincaré, *Sur le problème des trois corps et les équations de la dynamique*, Acta Mathematica **13** (1890) 1–270.
[23] E. Zermelo, *Über einen Satz der Dynamik und die mechanische Wärmetheorie*, Annalen der Physik **57** (1896) 485–494. doi:10.1002/andp.18962930314.
[24] J. M. Gandhi, *Formulae for the n-th prime*, in: Proceedings of the Washington State University Conference on Number Theory (1971) 96–106.
[25] S. W. Golomb, *Formulas for the Next Prime*, Pacific Journal of Mathematics **63**(2) (1976) 401–404. doi:10.2140/pjm.1976.63.401.
[26] J. B. Keller, *A Recursion Equation for Prime Numbers*, arXiv:0711.3940.
[27] B. Cloitre, *An Effective Analytic Recurrence for Prime Numbers*, arXiv:2508.02690v2 (2025).
[28] P. Cousot and R. Cousot, *Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints*, in: Proceedings of the 4th ACM SIGACT–SIGPLAN Symposium on Principles of Programming Languages (POPL '77), ACM, 1977, pp. 238–252. doi:10.1145/512950.512973.
[29] S. Graf and H. Saïdi, *Construction of Abstract State Graphs with PVS*, in: Computer Aided Verification (CAV 1997), Lecture Notes in Computer Science **1254**, Springer, 1997, pp. 72–83. doi:10.1007/3-540-63166-6_10.
[30] E. Clarke, O. Grumberg, S. Jha, Y. Lu and H. Veith, *Counterexample-Guided Abstraction Refinement for Symbolic Model Checking*, Journal of the ACM **50**(5) (2003) 752–794. doi:10.1145/876638.876643.
[31] B. Alpern and F. B. Schneider, *Defining Liveness*, Information Processing Letters **21**(4) (1985) 181–185. doi:10.1016/0020-0190(85)90056-0.

[32] Z. Manna and A. Pnueli, *A Hierarchy of Temporal Properties*, in: Proceedings of the Ninth Annual ACM Symposium on Principles of Distributed Computing (PODC '90), ACM, 1990, pp. 377–410. doi:10.1145/93385.93442.

[33] O. Kupferman and M. Y. Vardi, *Model Checking of Safety Properties*, Formal Methods in System Design **19** (2001) 291–314. doi:10.1023/A:1011254632723.

[34] R. W. Floyd, *Assigning Meanings to Programs*, in: Mathematical Aspects of Computer Science, Proceedings of Symposia in Applied Mathematics **19**, American Mathematical Society, 1967, pp. 19–32.

[35] A. Podelski and A. Rybalchenko, *A Complete Method for the Synthesis of Linear Ranking Functions*, in: Verification, Model Checking, and Abstract Interpretation (VMCAI 2004), Lecture Notes in Computer Science **2937**, Springer, 2004, pp. 239–251. doi:10.1007/978-3-540-24622-0_20.

[36] E. Bach, *Explicit Bounds for Primality Testing and Related Problems*, Mathematics of Computation **55**(191) (1990) 355–380. doi:10.1090/S0025-5718-1990-1023756-8.

[37] C. P. Willans, *On Formulae for the nth Prime Number*, The Mathematical Gazette **48** (1964) 413–415. doi:10.2307/3611701.

This work is licensed under the Creative Commons Attribution License. To view a copy of this license, visit https://creativecommons.org/licenses/by/4.0/ or send a letter to Creative Com-